

การออกแบบและสร้างแพลตฟอร์มไอโอทีเชิงอุตสาหกรรมเพื่อการศึกษา โดยผสาน
คอมพิวเตอร์บอร์ดเดี่ยว ไมโครคอนโทรลเลอร์ และพีแอลซี

นายพัทธนันท์ พันธุมณี

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมไฟฟ้า ภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568

ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Design and Implementation of an Educational Industrial IoT Platform using an
SBC, a MCU and a PLC

Mr. Patthanan Bhundhumani

THIS PROJECT IS A PARTIAL FULLFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF BACHELOR OF ELECTRICAL ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
ACADEMIC YEAR 2025
COPYRIGHT OF KING MONGKUT'S UNIVERSITY OF TECHNOLOGY
NORTH BANGKOK

ใบรับรองปริญญานิพนธ์

ชื่อปริญญานิพนธ์ : การออกแบบและสร้างแพลตฟอร์มไอโอทีเชิงอุตสาหกรรมเพื่อการศึกษา
โดยผสานคอมพิวเตอร์บอร์ดเดียว ไมโครคอนโทรลเลอร์ และพีแอลซี

ชื่อ : นายพัทธนันท์ พันธุมณี รหัสนักศึกษา 6301011610279

สาขาวิชา : วิศวกรรมไฟฟ้า

ภาควิชา : วิศวกรรมไฟฟ้าและคอมพิวเตอร์

คณะ : วิศวกรรมศาสตร์

ที่ปรึกษา : ดร.เรวัต ศิริโกศาภิรมย์

ปีการศึกษา : 2568

ได้รับอนุมัติให้นับเป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมไฟฟ้า ภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

.....
(รองศาสตราจารย์ ดร.แคทรียา สุวรรณศรี) หัวหน้าภาควิชาวิศวกรรมไฟฟ้า
และคอมพิวเตอร์

.....
(ดร.เรวัต ศิริโกศาภิรมย์) ประธานกรรมการ

.....
(ดร.ดนุชา ประเสริฐสม) กรรมการ

.....
(รองศาสตราจารย์ ดร.บัลลังก์ เนียมมณี) กรรมการ

ลิขสิทธิ์ของภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Approval Project Certificate

Project : Design and Implementation of an Educational Industrial IoT Platform using
an SBC, a MCU and a PLC

Name : Patthanan Bhundhumani ID.6301011610279

Major : Electrical Engineering

Department : Electrical and Computer Engineering

Faculty : Engineering

Project Advisors : Dr. Rawat Sriripokarpirom

Academic Years : 2025

Accepted by the Faculty of Engineering, King Mongkut's University of Technology North
Bangkok in Partial Fulfillment of the Requirements for the Degree of Bachelor of Electrical
Engineering.



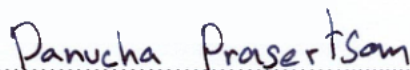
.....
(Assoc. Prof. Dr. Cattareeya Suwanasri)

Head of Electrical and Computer
Engineering Department



.....
(Dr. Rawat Sriripokarpirom)

Chairperson



.....
(Dr. Danucha Prasertsom)

Member



.....
(Assoc. Prof. Dr. Bunlung Neammanee)

Member

Copyright of the Department of Electrical and Computer Engineering, Faculty of Engineering,

King Mongkut's University of Technology North Bangkok

บทคัดย่อ

งานวิจัยนี้นำเสนอการออกแบบ การพัฒนา และการประเมินประสิทธิภาพของแพลตฟอร์ม ไอโอทีเชิงอุตสาหกรรมต้นทุนต่ำ สำหรับการเรียนการสอนระดับมหาวิทยาลัย โดยบูรณาการ อุปกรณ์ควบคุมสามประเภท ได้แก่ ไมโครคอนโทรลเลอร์ (MCU, ESP32-S3) คอมพิวเตอร์บอร์ด เดียว (SBC, Raspberry Pi 4) และพีแอลซี (PLC, Siemens LOGO! 24CE)

แพลตฟอร์มใช้ไมโครคอนโทรลเลอร์เป็นเกตเวย์แปลงโพรโทคอลจาก Modbus RTU เป็น Modbus TCP และรองรับ MQTT ได้ประเมินไลบรารีโอเพนซอร์สสำหรับเกตเวย์ Modbus จำนวน 8 รายการบนสโตร์เวอร์จิเนียตามเกณฑ์ที่กำหนด แล้ววัดสมรรถนะด้านเวลาแฝง อัตราการส่งข้อมูล และการใช้หน่วยความจำ

ชั้นไอทีใช้ซอฟต์แวร์โอเพนซอร์สที่นิยมใช้งานจริง ได้แก่ Mosquitto, Telegraf, InfluxDB, Grafana และ Node-RED ซึ่งลดการพึ่งพาซอฟต์แวร์เฉพาะของผู้ผลิตพีแอลซี

นอกจากนี้ยังพัฒนาเครื่องมือดักจับแพ็กเก็ตเพื่อสังเกตพฤติกรรม Modbus TCP และ MQTT บนเครือข่ายจริง ผลการทดลองแสดงว่าไมโครคอนโทรลเลอร์ราคาต่ำกว่า 2,000 บาทเพียง ตัวเดียวสามารถเชื่อมต่อระบบไอทีเดิมเข้ากับแดชบอร์ดไอทีสมัยใหม่ได้อย่างมีประสิทธิภาพ อุปกรณ์ควบคุมทั้งสามประเภทมีจุดแข็งเสริมกัน และเพิ่มประสิทธิภาพของระบบเมื่อใช้งาน ร่วมกัน

คำสำคัญ: ไอโอทีอุตสาหกรรม เกตเวย์ Modbus การผสมผสานไอทีและไอที ESP32 พีแอลซี

Abstract

This paper presents the design, development, and performance evaluation of a low-cost Industrial IoT (IIoT) platform for university teaching and laboratory practice. The platform integrates three control-device categories: a microcontroller unit (MCU, ESP32-S3), a single-board computer (SBC, Raspberry Pi 4), and a programmable logic controller (PLC, Siemens LOGO! 24CE).

The MCU was developed as a gateway converting Modbus RTU to Modbus TCP while also supporting MQTT. Eight open-source Modbus gateway libraries were evaluated on real hardware against predefined criteria, and the selected library was benchmarked for communication latency, data throughput, and memory usage.

The IT layer uses widely adopted open-source software — Mosquitto, Telegraf, InfluxDB, Grafana, and Node-RED — reducing dependence on proprietary PLC vendor software while remaining compatible with modern IIoT architectures.

A passive packet-capture tool was also developed to observe Modbus TCP and MQTT traffic on the live network during operation. Results show that a single microcontroller costing under THB 2,000 can effectively bridge legacy operational technology with modern IT dashboards, and that the three device categories offer complementary strengths whose combination improves overall system performance beyond any single platform alone.

Keywords: Industrial IoT, Modbus gateway, IT/OT convergence, ESP32, PLC

กิตติกรรมประกาศ

ผู้จัดทำปริญญานิพนธ์ขอขอบพระคุณทุกท่านที่มีส่วนช่วยเหลือและสนับสนุนในการทำ
รายงานโครงการนี้ ซึ่งเป็นรายงานสุดท้ายก่อนสำเร็จการศึกษา

ขอขอบพระคุณอาจารย์ที่ปรึกษา คร.เรวัต ศิริ โภคาภิรมย์ เป็นอย่างสูง สำหรับคำแนะนำ
และการสนับสนุนตลอดการทำงาน อาจารย์ไม่เพียงให้คำปรึกษาทางวิชาการที่ลึกซึ้งและตรง
ประเด็น หากยังสละเวลารับฟังปัญหา ชี้แนะแนวทางแก้ไข และให้ความสนใจในทุกสถานการณ์ที่
ผู้จัดทำปริญญานิพนธ์พบเจอ ความใส่ใจ ความอดทน และวิสัยทัศน์ของอาจารย์ เป็นทั้งแรงผลักดัน
และแบบอย่างที่ดีของการทำงานทางวิศวกรรม ปริญญานิพนธ์ฉบับนี้จะไม่อาจสำเร็จลุล่วงได้เลย
หากขาดการดูแลเอาใจใส่ของอาจารย์

สุดท้าย ผู้จัดทำปริญญานิพนธ์ขอขอบคุณครอบครัวที่เป็นผู้สนับสนุนที่ดีที่สุดตลอดชีวิต
ขอบคุณที่คอยให้กำลังใจและยื่นเคียงข้างผู้จัดทำปริญญานิพนธ์มาโดยตลอด

พัทธนันท์ พันธุมณี

สารบัญ

	หน้า
ใบรับรองปริญญานิพนธ์	ก
Approval Project Certificate	ข
บทคัดย่อ	ค
Abstract	ง
กิตติกรรมประกาศ	จ
สารบัญภาพ	ฉ
สารบัญตาราง	ฎ
ศัพท์เฉพาะ	ฏ
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและแรงจูงใจ	1
1.2 ความเป็นมาและความสำคัญของปัญหา	2
1.3 วัตถุประสงค์	2
1.4 ขอบเขตและข้อจำกัด	3
1.5 โครงสร้างของปริญญานิพนธ์	3
บทที่ 2 ความรู้พื้นฐานและงานวิจัยที่เกี่ยวข้อง	4
2.1 การผสมผสานไอที/โอที	4
2.2 ฮาร์ดแวร์และซอฟต์แวร์ที่พบทั่วไปในไอทีและโอที	4
2.2.1 ชุดเทคโนโลยีด้านเทคโนโลยีสารสนเทศ	4
2.2.2 ชุดเทคโนโลยีด้านเทคโนโลยีเชิงปฏิบัติการ	5
2.3 โพรโทคอลการสื่อสารในอุตสาหกรรม	6
2.3.1 Modbus RTU	6
2.3.2 Modbus TCP	7
2.3.3 MQTT	7

สารบัญ (ต่อ)

	หน้า
2.4 ประเภทของตัวควบคุม: MCU, SBC และ PLC	10
2.4.1 ไมโครคอนโทรลเลอร์ (MCU)	10
2.4.2 คอมพิวเตอร์บอร์ดเดี่ยว (SBC)	10
2.4.3 ตัวควบคุมโปรแกรมได้ (PLC)	11
2.5 ชุดเครื่องมือโอเพนซอร์สเทียบกับชุดเครื่องมือเฉพาะผู้ผลิต	11
2.6 ชุดแดชบอร์ดข้อมูลอนุกรมเวลา (TIG)	12
2.7 โพรโทคอล MQTT (Mosquitto)	12
2.8 Node-RED	12
2.9 งานวิจัยที่เกี่ยวข้องและช่องว่างงานวิจัย	13
บทที่ 3 การออกแบบและพัฒนาระบบ	15
3.1 สถาปัตยกรรมโดยรวม	15
3.2 การเลือกฮาร์ดแวร์ MCU, SBC และ PLC	16
3.3 ซอฟต์แวร์ชั้นไอที: การเขียนโปรแกรม PLC ด้วย LOGO! Soft Comfort	19
3.4 เฟิร์มแวร์เอดจ์เกตเวย์: Modbus บน ESP32-S3	23
3.5 ซอฟต์แวร์ชั้นไอที: MQTT และชุด TIG แบบคอนเทนเนอร์	24
3.6 เครื่องมือ IoT-Sniffer	25
3.7 การพัฒนาฟลอร์วอตโนมิตีด้วย Node-RED	26
บทที่ 4 การประเมินผลและผลการทดลอง	28
4.1 การประเมินไลบรารีเกตเวย์ Modbus	28
4.1.1 การตั้งค่าชุดทดลองและวิธีการ	28
4.1.2 ผลการทดลอง	30
4.1.3 ข้อเสนอแนะ	31
4.2 การวัดสมรรถนะของเกตเวย์	31

สารบัญ (ต่อ)

	หน้า
บทที่ 5 การอภิปรายผลและบทสรุป	37
5.1 ภาพรวมระบบ	37
5.2 ปัญหา อุปสรรค และวิธีการแก้ไข	38
5.3 ข้อเสนอแนะ	39
เอกสารอ้างอิง	40
ประวัติผู้แต่ง	41
ภาคผนวก ก ภาพรวมการออกแบบเฟิร์มแวร์ ESP32-S3	42
ภาคผนวก ข การกำหนดค่า Docker Compose และ Telegraf	44
ภาคผนวก ค ตารางคะแนนการประเมินไลบรารี Modbus	46
ภาคผนวก ง เครื่องมือ IoT-Sniffer และการใช้งานเพื่อตรวจสอบ	48
ภาคผนวก จ ข้อมูลอ้างอิงฮาร์ดแวร์	52
ภาคผนวก ฉ ตารางเปรียบเทียบแพลตฟอร์มไอไอโอที	57

สารบัญภาพ

ภาพที่		หน้า
2.1	สถาปัตยกรรมแบบผู้เผยแพร่-ผู้สมัครรับข้อความของ MQTT: โคลเอนด์เผยแพร่ข้อมูลไปยังหัวข้อบนโบรกเกอร์ และโบรกเกอร์กระจายต่อให้ผู้สมัครรับข้อความ	8
2.2	ลำดับการรับส่งข้อความที่ระดับ QoS 0 (at most once): ผู้ส่งเผยแพร่ข้อความเพียงครั้งเดียวโดยไม่มีการตอบรับ	8
2.3	ลำดับการรับส่งข้อความที่ระดับ QoS 1 (at least once): ผู้รับตอบกลับด้วยแพ็กเก็ต PUBACK และผู้ส่งเก็บข้อความไว้จนกว่าจะได้รับการตอบรับ	9
2.4	ลำดับการรับส่งข้อความที่ระดับ QoS 2 (exactly once): การจับมือสี่ขั้นตอนรับประกันว่าผู้รับได้รับข้อความเพียงครั้งเดียว	9
3.1	สถาปัตยกรรมโดยรวมของแพลตฟอร์ม: เครือข่ายอุตสาหกรรมแบบมีสาย เกตเวย์ Raspberry Pi 4 และอุปกรณ์ขอบไร้สาย ESP32-S3 (สร้างด้วย AI และปรับแก้บางส่วน)	16
3.2	ฮาร์ดแวร์จริงของแพลตฟอร์ม: เกตเวย์ ESP32-S3-Relay-6CH, XY-MD02 และ SDM230 พร้อมการเดินสาย RS-485	18
3.3	การติดตั้งฮาร์ดแวร์ของแพลตฟอร์ม: เกตเวย์ ESP32-S3-Relay-6CH ร่วมกับ XY-MD02 และ SDM230 (ซ้าย) และ Raspberry Pi 4 พร้อมจอสัมผัส (ขวา)	19
3.4	การเดินสายภายในแผงควบคุม Siemens LOGO! 24CE	20
3.5	แผงสวิตช์พีแอลซีที่ประกอบเสร็จ พร้อมสวิตช์ปุ่มกดและไฟแสดงสถานะ	21
3.6	ตัวอย่างโปรแกรมแลคเคอร์แบบล็อกตัวเอง (self-latching) สำหรับควบคุมเอาต์พุต Q1-Q3 ใน LOGO! Soft Comfort	21
3.7	หน้าต่าง Modbus address space ของ LOGO! Soft Comfort แสดงการแมปเอาต์พุต Q เป็น coil 8193-8212 และ V-memory เป็น holding registers	22
3.8	ผลการสั่งเปิดเอาต์พุตของพีแอลซีผ่าน Modbus TCP ด้วยคำสั่ง mbpoll: ไฟแสดงสถานะบนแผงสวิตช์ติดสว่างเมื่อเขียนค่า 1 ไปยัง coil 8193	23
3.9	เส้นทางการไหลของข้อมูลจากอุปกรณ์หน้างานผ่านเกตเวย์สู่ชั้นไอที (MBPoll, Node-RED, Mosquitto, Telegraf, InfluxDB และ Grafana) (สร้างด้วย AI และปรับแก้บางส่วน)	24
3.10	Raspberry Pi 4 ที่ติดตั้งใช้งานจริงพร้อมแหล่งจ่ายไฟและสายอีเทอร์เน็ต	25

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
3.11 ส่วนต่อประสานผู้ใช้ของเครื่องมือ IoT-Sniffer ขณะถอดรหัสคำขอ Modbus TCP Write Multiple Registers	26
3.12 โฟลว์ Node-RED สำหรับควบคุมลำดับการเปิดเอาต์พุต IO1-IO3 (LED sequencer) ด้วยปุ่ม Start/Stop	27
3.13 โฟลว์ Node-RED สำหรับอ่านค่ารีจิสเตอร์ HR0-HR3 จาก ESP32-S3 ผ่าน Modbus TCP (FC03) ทุก 5 วินาที	27
4.1 ชุดทดลองการศึกษา A บนบอร์ด: ESP32-S3 ทำหน้าที่เกตเวย์ Modbus RTU/TCP ร่วมกับตัวรับส่งสัญญาณ MAX3485 และ ESP32-C6 จำลองสเลฟ	29
4.2 แดชบอร์ด Grafana แสดงอุณหภูมิและความชื้นจาก XY-MD02 ที่รับผ่านเกตเวย์แบบเรียลไทม์	32
4.3 แดชบอร์ด Grafana แสดงค่าทางไฟฟ้าจากมิเตอร์ SDM230 (แรงดันไฟฟ้า กระแสไฟฟ้า และกำลังไฟฟ้าจริง)	33
4.4 แดชบอร์ดเฝ้าติดตามทรัพยากรของ Raspberry Pi (อุณหภูมิซีพียู โหลด และหน่วยความจำ) ที่เก็บผ่าน Telegraf	34
4.5 จอสัมผัสขนาด 7 นิ้วของ Raspberry Pi แสดงแดชบอร์ด Grafana ในโหมดคีย์บอร์ด	35
ง.1 มุมมองลำดับ (sequence view) แสดงการจับคู่คำขอและการตอบกลับ Modbus TCP ระหว่างอุปกรณ์บนเส้นเวลา	49
ง.2 มุมมองโทโพโลยี (topology view) แสดงการเชื่อมต่อระหว่างอุปกรณ์ในเครือข่าย แยกตามกลุ่มอุปกรณ์หน้างาน ห้องควบคุม โบรกเกอร์ เอจ และเซนเซอร์	50
ง.3 มุมมองเมทริกซ์ (matrix view) แสดงจำนวนแพ็กเก็ตที่แลกเปลี่ยนกันระหว่างอุปกรณ์แต่ละคู่	50
ง.4 แท็บสมรรถนะแสดงเวลาแฝงที่ p50/p95/p99 จิตเตอร์ และพฤติกรรมของโฟลว์แยกตามโปรโตคอล	51
จ.1 บอร์ด Waveshare ESP32-S3-Relay-6CH	53
จ.2 พีแอลซี Siemens LOGO! 24CE	54
จ.3 Raspberry Pi 4 Model B (2 GB)	54
จ.4 โมดูลเซนเซอร์วัดอุณหภูมิและความชื้น XY-MD02	55
จ.5 เพาเวอร์มิเตอร์ไฟฟ้าเฟสเดียว Eastron SDM230-Modbus	56

สารบัญตาราง

ตารางที่	หน้า
2.1 ชุดเทคโนโลยีตัวแทนในเทคโนโลยีสารสนเทศและเทคโนโลยีเชิงปฏิบัติการ	6
4.1 เวลาแฝงไปกลับของ FC03 และการใช้ทรัพยากรต่อไลบรารี Modbus (ESP32-S3, 100 รีจิสเตอร์, 9,600 baud) เวลาแฝงหน่วยเป็นมิลลิวินาที แฟลชและแรมหน่วยเป็นกิโลไบต์	30
5.1 การเปรียบเทียบตัวควบคุมทั้งสามประเภทที่ใช้ในแพลตฟอร์ม	37
ก.1 แผนผัง Holding Register (HR0-HR20) ของบทบาทสเลฟ TCP	43
ข.1 สรุปรหัสและบริการที่เปิดใช้งานบน Raspberry Pi	45
ค.1 คะแนนการประเมินเชิงคุณภาพของไลบรารีเกตเวย์ Modbus	46
ฉ.1 การเปรียบเทียบตัวควบคุม MCU, SBC และ PLC ในหกมิติเชิงวิศวกรรม	57

ศัพท์เฉพาะ

IIoT	ไอโอทีเชิงอุตสาหกรรม (Industrial Internet of Things)
IT / OT	เทคโนโลยีสารสนเทศ/เทคโนโลยีเชิงปฏิบัติการ
MCU	ไมโครคอนโทรลเลอร์ (microcontroller unit)
SBC	คอมพิวเตอร์บอร์ดเดียว (single-board computer)
PLC	ตัวควบคุมโปรแกรมได้ (programmable logic controller)
Modbus RTU / TCP	โพรโทคอลสื่อสารอุตสาหกรรมแบบร้องขอ-ตอบสนอง บนสายอนุกรมหรือ TCP/IP
MQTT	โพรโทคอลรับส่งข้อความแบบเผยแพร่-สมัครรับข้อความสำหรับ โทรมาตร
TIG	ชุดซอฟต์แวร์ Telegraf, InfluxDB และ Grafana
RS-485	มาตรฐานการสื่อสารอนุกรมแบบดิฟเฟอเรนเชียลสำหรับงาน อุตสาหกรรม
DMZ	เขตกันชนของเครือข่ายระหว่างชั้นไอทีและโอที (demilitarized zone)

บทที่ 1

บทนำ

1.1 ความเป็นมาและแรงจูงใจ

แนวคิดอุตสาหกรรม 4.0 (Industry 4.0) ผลักดันให้โรงงานนำข้อมูลจากหน้างานขึ้นไปใช้บนระบบเทคโนโลยีสารสนเทศ (Information Technology: IT) เพื่อเฝ้าติดตาม วิเคราะห์ และควบคุมดูแลจากระยะไกล อย่างไรก็ตามอุปกรณ์ในฝั่งเทคโนโลยีเชิงปฏิบัติการ (Operational Technology: OT) ที่ควบคุมกระบวนการทางกายภาพ ส่วนใหญ่ยังคงสื่อสารด้วยโปรโตคอลที่ใช้กันมานานหลายสิบปี ตัวอย่างที่พบได้ทั่วไปคือ Modbus ซึ่งเริ่มใช้งานมาตั้งแต่ปี ค.ศ. 1979 [4] โดยรูปแบบอนุกรม (Modbus RTU) สื่อสารผ่านบัส RS-485 และไม่มีช่องทางเชื่อมต่อเข้ากับเครือข่ายไอพี (IP network) มาให้ในตัว ในทางปฏิบัติการรื้อเปลี่ยนอุปกรณ์หน้างานเดิมทั้งหมดก็ไม่คุ้มค่าในเชิงเศรษฐศาสตร์ ดังนั้นโจทย์ที่แท้จริงจึงไม่ใช่การเปลี่ยนอุปกรณ์ใหม่ แต่เป็นการเชื่อมของเก่าเข้ากับของใหม่ กล่าวคือ ต้องนำข้อมูลจากฝั่งไอทีเดิมขึ้นไปแสดงผลบนแดชบอร์ดของฝั่งไอทีสมัยใหม่ให้ได้ด้วยต้นทุนต่ำ ทั้งนี้การดำเนินการดังกล่าวต้องไม่กระทบต่อความแน่นอนเชิงเวลา (determinism) ที่งานฝั่งไอทีต้องรักษาไว้

ในบริบทดังกล่าวมีตัวควบคุม (controller) อยู่สามประเภทหลักที่นักศึกษาซึ่งเข้าสู่สาขานี้จำเป็นต้องเข้าใจ ได้แก่ ไมโครคอนโทรลเลอร์ (Microcontroller: MCU) คอมพิวเตอร์บอร์ดเดี่ยว (Single-Board Computer: SBC) และตัวควบคุมแบบโปรแกรมได้ หรือพีแอลซี (Programmable Logic Controller: PLC) ทั้งสามประเภทนี้แตกต่างกันไม่เพียงด้านหน่วยประมวลผลและพอร์ต I/O เท่านั้น แต่ยังต่างกันในเรื่องที่งานนี้ให้ความสำคัญเป็นลำดับแรก นั่นคือระบบนิเวศของซอฟต์แวร์ (software ecosystem) โดย MCU และ SBC พัฒนาได้ด้วยซอฟต์แวร์โอเพนซอร์สที่เป็นที่รู้จักกันดี เช่น Arduino core ร่วมกับ PlatformIO, Linux ร่วมกับ Docker, ชุดซอฟต์แวร์ Telegraf-InfluxDB-Grafana (หรือชุด “TIG”), Node-RED และโบรกเกอร์ MQTT อย่าง Mosquitto ในทางกลับกัน พีแอลซีมักผูกติดกับชุดเครื่องมือเฉพาะของผู้ผลิต ซึ่งในกรณีของ Siemens LOGO! ที่เลือกใช้ในงานนี้คือซอฟต์แวร์ LOGO! Soft Comfort ความแตกต่างด้านความเปิดกว้างของซอฟต์แวร์นี้เองที่เป็นบทเรียนสำคัญเรื่องการผูกติดกับผู้ผลิต (vendor lock-in) ในฝั่งไอที

1.2 ความเป็นมาและความสำคัญของปัญหา

การเรียนการสอนเรื่องการผสมผสานไอที/โอทีให้เกิดผลจริงจำเป็นต้องมีฮาร์ดแวร์ที่นักศึกษาได้ลงมือสัมผัส ทดลองจนเกิดข้อผิดพลาด และตรวจวัดผลได้ด้วยตนเอง แต่ชุดฝึกไอโอไอที (IIoT) เชิงพาณิชย์ที่มีอยู่ มักมีราคาสูงและเป็นระบบปิด จึงบดบังพฤติกรรมระดับสายสัญญาณ (wire-level) ที่ผู้เรียนควรได้เห็น จากสภาพเช่นนี้จึงเกิดช่องว่างขึ้นสองประการ ประการแรก ยังไม่มีชุดทดลองต้นทุนต่ำที่นำเสนอตัวควบคุมครบทั้งสามประเภทควบคู่ไปกับโปรโตคอลอุตสาหกรรมจริงในชุดเดียวกัน ประการที่สอง แม้จะมีไลบรารีสำหรับสร้างเกตเวย์ Modbus แบบโอเพนซอร์สบน ESP32 ให้เลือกอยู่จำนวนมาก แต่กลับยังไม่มีเปรียบเทียบเชิงประจักษ์ที่ช่วยชี้แนะว่าควรเลือกใช้ไลบรารีใด ผู้พัฒนาจึงต้องตัดสินใจเลือกโดยขาดข้อมูลอ้างอิง

1.3 วัตถุประสงค์

1.3.1 ออกแบบและสร้างแพลตฟอร์มห้องปฏิบัติการไอโอทีเชิงอุตสาหกรรมต้นทุนต่ำเพื่อการศึกษา ที่บูรณาการฮาร์ดแวร์หลักสามประเภทของงานอัตโนมัติสมัยใหม่เข้าด้วยกัน ได้แก่ คอมพิวเตอร์บอร์ดเดี่ยว (SBC, Raspberry Pi 4), ไมโครคอนโทรลเลอร์ (MCU, ESP32-S3) และพีแอลซี (PLC, Siemens LOGO! 24CE)

1.3.2 สาธิตการเชื่อมประสานระหว่างอุปกรณ์ฝั่งไอทีและฝั่งโอที โดยเน้นการใช้ชุดซอฟต์แวร์โอเพนซอร์ส ด้วยการนำเครื่องมือที่นิยมใช้กันแพร่หลายในงานไอโอไอที ได้แก่ โบรกเกอร์ MQTT (Mosquitto), Node-RED และชุด TIG (Telegraf, InfluxDB, Grafana) มาทำหน้าที่เป็นชั้นขอบ (edge) และชั้นไอทีของแพลตฟอร์ม

1.3.3 พัฒนาเกตเวย์แปลงโปรโตคอลจาก Modbus RTU ไปเป็น TCP และ MQTT บนไมโครคอนโทรลเลอร์ พร้อมทั้งประเมินไลบรารีเกตเวย์ Modbus แบบโอเพนซอร์สบน ESP32-S3 โดยทดสอบสมรรถนะของไลบรารีที่รองรับการทำงานเป็นเกตเวย์บนฮาร์ดแวร์จริง เพื่อคัดเลือกไลบรารีที่เหมาะสมที่สุดหนึ่งตัว จากนั้นวัดเวลาแฝง อัตราการส่งข้อมูล และปริมาณการใช้หน่วยความจำแฟลชและแรมของเกตเวย์ที่ได้

1.3.4 พัฒนาเครื่องมือ IoT-Sniffer สำหรับถอดรหัสข้อมูล Modbus TCP และ MQTT เพื่อสังเกตกิจกรรมที่เกิดขึ้นบนเครือข่ายและวัดสมรรถนะเครือข่ายของอุปกรณ์ไอโอทีในแพลตฟอร์ม

1.4 ขอบเขตและข้อจำกัด

แพลตฟอร์มนี้จัดทำขึ้นเป็นชุดทดลองที่ทำงานบนเครือข่ายท้องถิ่น (Local Area Network: LAN) ภายในสถานที่เดียว ขอบเขตของงานครอบคลุม Modbus ทั้งแบบ RTU และ TCP, โพรโทคอล MQTT, ชุดซอฟต์แวร์ TIG และเครื่องมือดักจับข้อมูลแบบเชิงรับ (passive) ส่วนหัวข้อที่อยู่นอกขอบเขต ได้แก่ โพรโทคอล OPC-UA การรองรับมาตรฐานอุตสาหกรรม การติดตั้งใช้งานในพื้นที่อันตราย (ATEX) และการเตรียมความพร้อมสำหรับใช้งานจริงในเชิงการผลิต ทั้งนี้ผลการประเมินไลบรารีได้มาจากการทดสอบบนฮาร์ดแวร์ ESP32-S3 เพียงรุ่นเดียว จึงอาจให้ผลแตกต่างออกไปเมื่อใช้กับฮาร์ดแวร์รุ่นอื่น

1.5 โครงสร้างของปริญญานิพนธ์

เนื้อหาของปริญญานิพนธ์นี้แบ่งออกเป็นห้าบท บทที่ 2 ทบทวนความรู้พื้นฐานเรื่องการผสมผสานไอที/โอที ฮาร์ดแวร์และซอฟต์แวร์ที่พบได้ทั่วไปในทั้งสองฝั่ง โพรโทคอล Modbus และ MQTT ตัวควบคุมทั้งสามประเภทพร้อมการเปรียบเทียบชุดเครื่องมือโอเพนซอร์สกับชุดเครื่องมือเฉพาะของผู้ผลิต ชุดแดชบอร์ดสำหรับแสดงผล ตลอดจนงานวิจัยที่เกี่ยวข้อง บทที่ 3 นำเสนอสถาปัตยกรรมของแพลตฟอร์มและรายละเอียดการพัฒนาในแต่ละชั้น ทั้งเฟิร์มแวร์เกตเวย์บน MCU ชุดซอฟต์แวร์ฝั่งไอที และเครื่องมือดักจับข้อมูล บทที่ 4 รายงานผลการประเมิน ได้แก่ การทดสอบไลบรารี Modbus และการวัดสมรรถนะของเกตเวย์ ส่วนบทที่ 5 อภิปรายผลการศึกษา ความเหมาะสมของตัวควบคุมแต่ละประเภท ข้อจำกัดของงาน ตลอดจนแนวทางการพัฒนาต่อในอนาคต

บทที่ 2

ความรู้พื้นฐานและงานวิจัยที่เกี่ยวข้อง

บทนี้ปูพื้นฐานความรู้เชิงทฤษฎีที่จำเป็นต่อการอธิบายเหตุผลเบื้องหลังการออกแบบแพลตฟอร์ม โดยเริ่มจากความแตกต่างระหว่างฝั่งไอทีและฝั่งโอที แล้วสำรวจฮาร์ดแวร์และซอฟต์แวร์ที่พบได้ทั่วไปในแต่ละฝั่ง ตามด้วยโพรโทคอลที่ใช้ในงานอุตสาหกรรม ตัวควบคุมทั้งสามประเภทและระบบนิเวศซอฟต์แวร์ของแต่ละประเภท ตลอดจนชุดแคชบอร์ดสำหรับแสดงผลข้อมูล และปิดท้ายด้วยการทบทวนงานวิจัยที่เกี่ยวข้องพร้อมชี้ให้เห็นช่องว่างที่งานนี้มุ่งเข้าไปแก้ไข

2.1 การผสมผสานไอที/โอที

งานอัตโนมัติในภาคอุตสาหกรรมแบ่งเทคโนโลยีออกเป็นสองฝั่งมาโดยตลอด ฝั่งเทคโนโลยีเชิงปฏิบัติการ (Operational Technology: OT) ประกอบด้วยตัวควบคุม เช่น เซอร์ และ แอกชูเอเตอร์ ที่ทำหน้าที่ขับเคลื่อนกระบวนการทางกายภาพ จึงให้ความสำคัญกับความแน่นอนเชิงเวลา ความพร้อมใช้งาน และความปลอดภัยเป็นหลัก ส่วนฝั่งเทคโนโลยีสารสนเทศ (Information Technology: IT) ประกอบด้วยเซิร์ฟเวอร์ ฐานข้อมูล และแคชบอร์ดที่ทำหน้าที่จัดเก็บและแสดงผลข้อมูล จึงให้ความสำคัญกับอัตราการส่งข้อมูล ความยืดหยุ่น และการรักษาความลับ แนวคิดอุตสาหกรรม 4.0 ต้องการให้ข้อมูลจากฝั่งไอทีไหลไปถึงระบบฝั่งโอทีได้แบบเรียลไทม์ ซึ่งเป้าหมายนี้เองที่เรียกกันว่า การผสมผสานไอที/โอที (IT/OT convergence) [1], [2]

2.2 ฮาร์ดแวร์และซอฟต์แวร์ที่พบทั่วไปในไอทีและโอที

นอกเหนือจากความแตกต่างในเชิงแนวคิดที่กล่าวถึงในหัวข้อ 2.1 แล้ว ฝั่งไอทีและฝั่งโอทียังถูกสร้างขึ้นจากชุดเทคโนโลยีที่แตกต่างกันอย่างชัดเจน หากระบุให้ชัดเจนว่าแต่ละฝั่งประกอบด้วยเทคโนโลยีใดบ้าง ก็จะช่วยให้เข้าใจการตัดสินใจออกแบบต่าง ๆ ในปริญญานิพนธ์นี้ได้ง่ายขึ้น

2.2.1 ชุดเทคโนโลยีด้านเทคโนโลยีสารสนเทศ

โครงสร้างพื้นฐานของฝั่งไอที มีแกนหลักอยู่ที่การประมวลผลแบบเนกประสงค์ และการจัดการข้อมูลระดับองค์กร [1], [2] ฮาร์ดแวร์ที่พบทั่วไป ได้แก่ เซิร์ฟเวอร์แบบติดตั้งในตู้ rack ระบบจัดเก็บข้อมูลแบบ SAN หรือ NAS สวิตช์และเราเตอร์อีเทอร์เน็ต และเวิร์กสเตชันของ

ผู้ใช้ อุปกรณ์เหล่านี้ส่วนใหญ่เป็นสถาปัตยกรรม x86 ทั่วไป และถูกเปลี่ยนแปลงตามรอบทุกสามถึงห้าปี ในชั้นซอฟต์แวร์นิยมใช้ระบบปฏิบัติการอเนกประสงค์ เช่น Windows Server หรือ Linux ซึ่งมีทำงานบนเครื่องเสมือน (virtual machine) ที่จัดการด้วยไฮเปอร์ไวเซอร์ (hypervisor) อย่าง VMware ESXi หรือ Microsoft Hyper-V ส่วนด้านฐานข้อมูลนิยมใช้ฐานข้อมูลเชิงสัมพันธ์หรือฐานข้อมูลเอกสาร เช่น Microsoft SQL Server, PostgreSQL หรือ Oracle Database เพื่อรองรับแอปพลิเคชันองค์กร เช่น ระบบวางแผนทรัพยากรองค์กร (ERP) ระบบบริหารการผลิต และระบบบริหารความสัมพันธ์ลูกค้าอย่าง SAP หรือ Oracle E-Business Suite ในด้านการสื่อสาร เครือข่ายฝั่งไอทีตั้งอยู่บนชุดโพรโทคอล TCP/IP โดยรับส่งข้อมูลผ่าน HTTP/HTTPS และ REST เป็นหลัก และรักษาความปลอดภัยด้วย TLS ไฟร์วอลล์ และการจัดการอัตลักษณ์แบบรวมศูนย์ โดยรวมแล้วเทคโนโลยีฝั่งไอทีได้รับการอัปเดตแพตช์บ่อย ทนต่อการรื้อสตาร์ท และออกแบบมาเพื่ออัตราการส่งข้อมูล ความยืดหยุ่น และการรักษาความลับของข้อมูล มากกว่าความแน่นอนเชิงเวลา [3]

2.2.2 ชุดเทคโนโลยีด้านเทคโนโลยีเชิงปฏิบัติการ

ในทางกลับกัน โครงสร้างพื้นฐานของฝั่งไอทีมุ่งเน้นฮาร์ดแวร์สำหรับกระบวนการ และการควบคุมที่ต้องการความแน่นอนเชิงเวลา ฮาร์ดแวร์ระดับหน้างานประกอบด้วย เซนเซอร์และแอคชูเอเตอร์ หน่วยปลายทางระยะไกล (Remote Terminal Unit: RTU) ที่ติดตั้งกระจายตามพื้นที่ และตัวควบคุมแบบโปรแกรมได้ (PLC) สำหรับกระบวนการต่อเนื่องขนาดใหญ่ จะใช้ระบบควบคุมแบบกระจาย (Distributed Control System: DCS) ซึ่งประสานการทำงานของตัวควบคุมหลายตัวจากสถานีวิศวกรรมส่วนกลาง ผู้ปฏิบัติงานได้ตอบกับกระบวนการผ่านส่วนต่อประสานระหว่างมนุษย์กับเครื่องจักร (Human-Machine Interface: HMI) ส่วนซอฟต์แวร์ SCADA ทำหน้าที่รวบรวม HMI และตัวควบคุมจำนวนมากทั่วทั้งไซต์งาน ขณะที่ฮิสทอรีน (historian) ทำหน้าที่บันทึกค่าของกระบวนการไว้วิเคราะห์แนวโน้มระยะยาว แพลตฟอร์มเชิงพาณิชย์ที่เป็นตัวแทนของฝั่งนี้ ได้แก่ Siemens SIMATIC S7/WinCC ร่วมกับ TIA Portal, Rockwell Automation Allen-Bradley ControlLogix/Studio 5000 ร่วมกับ FactoryTalk และ Schneider Electric Modicon ร่วมกับ EcoStruxure รวมถึงซอฟต์แวร์ SCADA และฮิสทอรีนที่ไม่ผูกกับผู้ผลิตรายใดรายหนึ่ง เช่น Inductive Automation Ignition, AVEVA (Wonderware) System Platform และ PI Historian ด้านการสื่อสาร ฝั่งไอทีอาศัยโพรโทคอลหน้างานที่ออกแบบมาสำหรับเครือข่ายอนุกรมหรือเครือข่ายที่ต้องการความแน่นอนเชิงเวลา โดยเฉพาะ Modbus (หัวข้อ 2.3.1-2.3.2), PROFIBUS/PROFINET และ DNP3 ส่วน OPC UA และ MQTT (มักใช้ผ่านข้อกำหนด Sparkplug B) เริ่มมีบทบาทมากขึ้นในการเชื่อมข้อมูลฝั่งไอทีเข้ากับระบบฝั่งไอที ทั้งนี้เนื่องจากความผิดพลาดในการควบคุมอาจทำให้อุปกรณ์เสียหาย หรือเป็นอันตรายต่อผู้ปฏิบัติงาน ฮาร์ดแวร์ฝั่งไอทีจึงถูก

ออกแบบให้ทำงานต่อเนื่องตลอดเวลา เปลี่ยนอุปกรณ์เฉพาะช่วงหยุดซ่อมบำรุงตามกำหนด และมีอายุการใช้งานยาวนานสิบถึงยี่สิบปี โดยให้น้ำหนักกับความพร้อมใช้งาน ความปลอดภัย และความแน่นอนเชิงเวลา มากกว่าอัตราการส่งข้อมูลหรือความถี่ในการอัปเดตแพตช์ [1]

ตารางที่ 2.1 ชุดเทคโนโลยีตัวแทนในเทคโนโลยีสารสนเทศและเทคโนโลยีเชิงปฏิบัติการ

หมวดหมู่	เทคโนโลยีสารสนเทศ (IT)	เทคโนโลยีเชิงปฏิบัติการ (OT)
ฮาร์ดแวร์ทั่วไป	เซิร์ฟเวอร์เร็ก, ที่จัดเก็บข้อมูล SAN/NAS, สวิตช์/เราเตอร์อีเทอร์ เน็ต, เวอร์กสแตชัน	เซนเซอร์/แอ็กชูเอเตอร์, RTU, PLC, ตัวควบคุม DCS, แผง HMI
สภาพแวดล้อมการทำงาน	Windows Server / Linux, เครื่อง เสมือนบนไฮเปอร์- ไวเซอร์	เฟิร์มแวร์ของผู้ผลิตหรือ RTOS, รอบสแกนที่มีความแน่นอนเชิง เวลา
ซอฟต์แวร์แอปพลิเคชัน	ERP/MES/CRM (เช่น SAP, Oracle), ฐานข้อมูล SQL/NoSQL	SCADA (Ignition, WinCC, FactoryTalk, System Platform), ซอฟต์แวร์วิศวกรรม (TIA Portal, Studio 5000, LOGO! Soft Comfort), ฮิสทอรีน (PI Historian)
โพรโทคอลเครือข่าย	TCP/IP, HTTP/HTTPS, REST	Modbus, PROFIBUS/PROFINET, DNP3, OPC UA, MQTT (Sparkplug B)
รอบการทดแทน/อายุการใช้งาน	3-5 ปี อัปเดตแพตช์บ่อยครั้ง	10-20 ปี อัปเดตแพตช์เฉพาะช่วง หยุดตามกำหนด
ลำดับความสำคัญหลัก	อัตราการส่งข้อมูล, ความยืดหยุ่น, การรักษาความลับ	ความพร้อมใช้งาน, ความปลอดภัย , ความแน่นอนเชิงเวลา

2.3 โพรโทคอลการสื่อสารในอุตสาหกรรม

2.3.1 Modbus RTU

Modbus เป็นโพรโทคอลสำหรับการสื่อสารข้อมูลในงานอุตสาหกรรม เผยแพร่โดยบริษัท Modicon ตั้งแต่ปี ค.ศ. 1979 ทำงานในรูปแบบร้องขอ-ตอบสนอง (request-response) โดยมีอุปกรณ์ที่เป็นมาสเตอร์ (master) ทำหน้าที่สำรวจ (poll) อุปกรณ์สเลฟ (slave) ตั้งแต่หนึ่งตัวขึ้นไป [4] ในรูปแบบ Modbus RTU ข้อมูลจะถูกส่งผ่านบัสอนุกรม RS-485 โดยแต่ละเฟรมข้อมูลประกอบด้วยหมายเลขอุปกรณ์สเลฟ (slave address) ขนาดหนึ่งไบต์ รหัสคำสั่ง (function code)

ขนาดหนึ่งไบต์ ฟิลด์ข้อมูล และค่าตรวจสอบความถูกต้องแบบ 16-bit CRC (cyclic redundancy check) ขนาดสองไบต์ รหัสคำสั่ง (Function Code, FC) ใช้เลือกกลุ่มของข้อมูลที่ต้องการเข้าถึง เช่น FC=0x01/0x05 สำหรับ discrete coils, FC=0x02 สำหรับ discrete inputs, FC=0x03/0x06/0x10 สำหรับ holding registers และ FC=0x04 สำหรับ input registers โดยรีจิสเตอร์แต่ละตัวมีขนาด 2 ไบต์ (16 บิต) และเข้ารหัสแบบ big-endian ทั้งนี้ Modbus RTU ไม่มีการระบุที่อยู่นอกเหนือจากหมายเลขสเลฟ จึงเป็นโพรโทคอลที่เรียบง่ายและนำไปใช้งานได้ง่าย

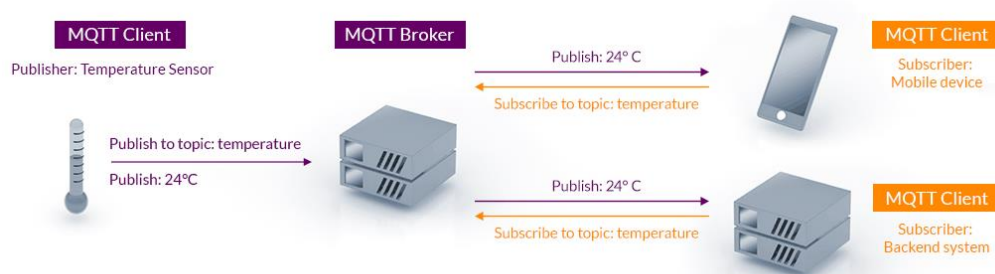
2.3.2 Modbus TCP

Modbus TCP นำหน่วยข้อมูลชุดเดียวกันกับ Modbus RTU มาส่งผ่านเครือข่าย TCP/IP ที่พอร์ตหมายเลข 502 โดยแทนที่เฟรมอนุกรมด้วยส่วนหัวที่เรียกว่า MBAP (Modbus Application Protocol Header) ขนาดเจ็ดไบต์ ซึ่งประกอบด้วยฟิลด์ Transaction ID สำหรับจับคู่คำขอกับการตอบกลับ หมายเลขโพรโทคอล (protocol identifier ซึ่งมีค่าเป็นศูนย์เสมอ) ฟิลด์ความยาว (length) และหมายเลขหน่วย (unit identifier) [4] ในกรณีนี้ไม่จำเป็นต้องมีค่า CRC เนื่องจากโพรโทคอล TCP รับประกันความถูกต้องของข้อมูลอยู่แล้ว เนื่องจากรหัสคำสั่งและแบบจำลองรีจิสเตอร์ยังคงเหมือนเดิม อุปกรณ์ที่รองรับทั้งสองรูปแบบจึงสามารถทำหน้าที่เป็นเอดจ์เกตเวย์ (edge gateway) แปลงโพรโทคอลระหว่าง Modbus RTU และ Modbus TCP เพื่อเปิดให้อุปกรณ์สเลฟแบบอนุกรมเข้าถึงได้จากโฮสต์ใด ๆ บนเครือข่ายไอพี ความเข้ากันได้นี้เองเป็นพื้นฐานของเอดจ์เกตเวย์ที่พัฒนาขึ้นในบทที่ 3

2.3.3 MQTT

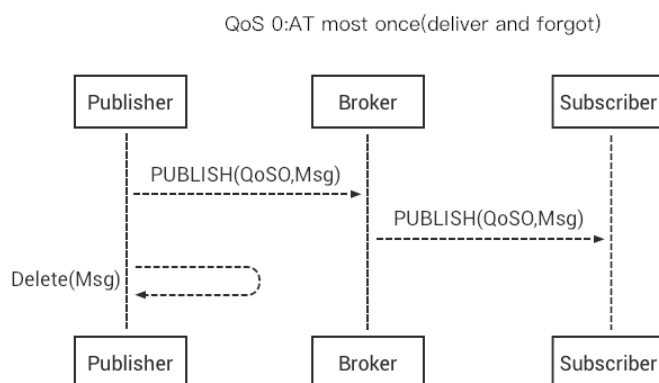
MQTT (เอ็ม-คิว-ที-ที) ย่อมาจาก MQ Telemetry Transport เป็นโพรโทคอลสำหรับการส่งข้อความ (messaging protocol) ที่มีน้ำหนักเบา ทำงานอยู่บนโพรโทคอล TCP/IP และเหมาะสำหรับงานไอโอที ได้รับการกำหนดให้เป็นมาตรฐาน ISO/IEC 20922 [5] สถาปัตยกรรมของ MQTT เป็นไปตามรูปแบบผู้เผยแพร่-ผู้สมัครรับข้อความ (publisher-subscriber pattern) ประกอบด้วยสองส่วนหลัก คือ โบรกเกอร์ (broker หรือตัวกลาง) ทำหน้าที่คอยรับและกระจายข้อความ และไคลเอนต์ (client) ซึ่งเป็นผู้ส่งหรือรับข้อความ ไคลเอนต์ที่เป็นผู้เผยแพร่ข้อความ (publisher) จะส่งข้อความไปยังหัวข้อ (topic) บนโบรกเกอร์ และโบรกเกอร์จะกระจายข้อความต่อไปยังไคลเอนต์ทุกตัวที่สมัครรับข้อความ (subscriber) ในหัวข้อนั้น ทำให้ผู้ส่งและผู้รับข้อมูลแยกออกจากกันและไม่ต้องเชื่อมต่อกันโดยตรง การส่งข้อความสามารถระบุระดับคุณภาพการให้บริการ (Quality of Service, QoS) ได้สามระดับ คือ QoS 0 (at most once) ส่งอย่างมากหนึ่งครั้งโดยไม่รับประกันการได้รับ QoS 1 (at least once) ส่งอย่างน้อยหนึ่งครั้งพร้อมการตอบกลับด้วย

แพ็คเกจ PUBACK และ QoS 2 (exactly once) รับประกันว่าผู้รับได้รับเพียงครั้งเดียว สถาปัตยกรรมดังกล่าวแสดงในภาพที่ 2.1

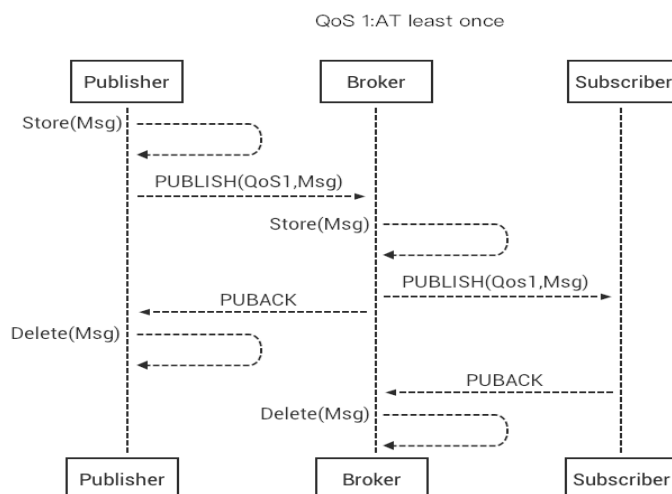


ภาพที่ 2.1 สถาปัตยกรรมแบบผู้เผยแพร่-ผู้สมัครรับข้อความของ MQTT: โคลเอนด์เผยแพร่ข้อมูลไปยังหัวข้อนบนโบรกเกอร์ และโบรกเกอร์กระจายต่อให้ผู้สมัครรับข้อความ (ที่มา: mqtt.org)

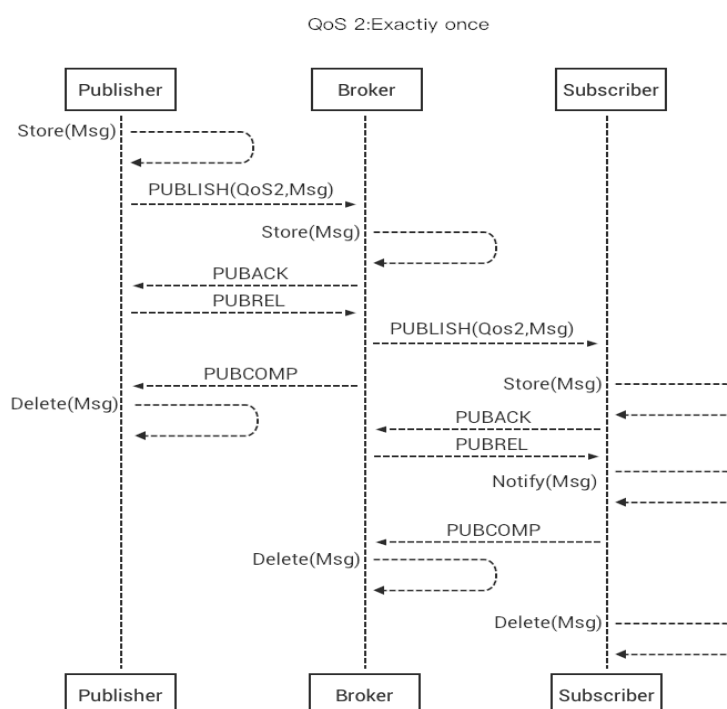
ลำดับการแลกเปลี่ยนข้อความระหว่างผู้เผยแพร่ โบรกเกอร์ และผู้สมัครรับข้อความในแต่ละระดับ QoS แสดงในภาพที่ 2.2 ถึงภาพที่ 2.4



ภาพที่ 2.2 ลำดับการรับส่งข้อความที่ระดับ QoS 0 (at most once): ผู้ส่งเผยแพร่ข้อความเพียงครั้งเดียวโดยไม่มีการตอบรับ (ที่มา: EMQX Enterprise Documentation)



ภาพที่ 2.3 ลำดับการรับส่งข้อความที่ระดับ QoS 1 (at least once): ผู้รับตอบกลับด้วยแพ็กเก็ต PUBACK และผู้ส่งเก็บข้อความไว้จนกว่าจะได้รับการตอบรับ (ที่มา: EMQX Enterprise Documentation)



ภาพที่ 2.4 ลำดับการรับส่งข้อความที่ระดับ QoS 2 (exactly once): การจับมือสั่นตอนรับประกันว่าผู้รับได้รับข้อความเพียงครั้งเดียว (ที่มา: EMQX Enterprise Documentation)

นอกจากนี้ยังมีคุณลักษณะข้อความที่เก็บรักษาไว้ (retained message) และการส่งข้อความครั้งสุดท้าย (last will and testament) เพื่อรองรับการเชื่อมต่อที่ไม่ต่อเนื่อง โดยคำเริ่มต้นโพรโทคอล MQTT อนุญาตให้เข้าถึงแบบไม่ระบุตัวตน ซึ่งช่วยให้การติดตั้งใช้งานในห้องเรียนทำได้ง่าย

2.4 ประเภทของตัวควบคุม: MCU, SBC และ PLC

อุปกรณ์ควบคุมแบบโปรแกรมได้ที่ใช้ในงานอัตโนมัติยุคปัจจุบันแบ่งออกได้เป็นสามประเภทหลัก คือ ไมโครคอนโทรลเลอร์ (MCU) คอมพิวเตอร์บอร์ดเดี่ยว (SBC) และตัวควบคุมแบบโปรแกรมได้หรือพีแอลซี (PLC) การศึกษาที่ครบถ้วนจึงจำเป็นต้องเข้าใจทั้งสามประเภทควบคู่กันไป หัวข้อนี้จะอธิบายลักษณะโดยรวมของตัวควบคุมแต่ละประเภท พร้อมระบุอุปกรณ์เฉพาะรุ่นที่งานนี้เลือกนำมาใช้ ส่วนรายละเอียดของการเลือกและการนำไปใช้งานจริงจะกล่าวถึงในบทที่ 3

2.4.1 ไมโครคอนโทรลเลอร์ (MCU)

ไมโครคอนโทรลเลอร์ (Microcontroller: MCU) คือชิปที่รวมหน่วยประมวลผล หน่วยความจำ และอุปกรณ์รอบข้างต่าง ๆ ไว้ในชิปตัวเดียว จึงเหมาะกับงานที่ต้องควบคุมสัญญาณระดับต่ำและตอบสนองแบบเรียลไทม์ในระดับหนึ่ง โดยทั่วไปมีต้นทุนและการใช้พลังงานต่ำ มีพอร์ตสื่อสารอนุกรมและพอร์ต I/O แบบดิจิทัลให้เชื่อมต่อกับเซนเซอร์และอุปกรณ์หน้างานได้โดยตรง แต่ก็มีหน่วยความจำและกำลังประมวลผลจำกัด และมักไม่มีระบบปฏิบัติการเต็มรูปแบบในงานนี้เลือกใช้ไมโครคอนโทรลเลอร์ที่ใช้ชิป Espressif ESP32-S3 บนบอร์ด Waveshare ESP32-S3-Relay-6CH ซึ่งมีทั้ง Wi-Fi และอินเทอร์เฟซ RS-485 มาให้ในตัว (รายละเอียดของบอร์ดอยู่ในหัวข้อ 3.2 และภาคผนวก จ)

2.4.2 คอมพิวเตอร์บอร์ดเดี่ยว (SBC)

คอมพิวเตอร์บอร์ดเดี่ยว (Single-Board Computer: SBC) คือคอมพิวเตอร์ขนาดเล็กที่รวมทุกส่วนไว้บนแผ่นวงจรเดียว มีตัวประมวลผลแบบหลายแกนและหน่วยความจำมากพอที่จะรันระบบปฏิบัติการเต็มรูปแบบอย่าง Linux ได้ จึงมีความสามารถและความยืดหยุ่นด้านซอฟต์แวร์สูงกว่าไมโครคอนโทรลเลอร์มาก เหมาะกับงานประมวลผลข้อมูล การจัดเก็บข้อมูล และการทำหน้าที่เป็นเซิร์ฟเวอร์ขนาดเล็ก แต่เนื่องจากใช้ตัวจัดการงานแบบอเนกประสงค์ จึงไม่สามารถรับประกันการตอบสนองแบบเรียลไทม์ที่เข้มงวดได้ ในงานนี้เลือกใช้คอมพิวเตอร์บอร์ดเดี่ยวรุ่น Raspberry Pi 4 Model B มาทำหน้าที่เป็นชั้นไอทีของแพลตฟอร์ม (รายละเอียดอยู่ในหัวข้อ 3.2 และภาคผนวก จ)

2.4.3 ตัวควบคุมโปรแกรมได้ (PLC)

ตัวควบคุมแบบโปรแกรมได้ (Programmable Logic Controller: PLC) คือ คอมพิวเตอร์สำหรับงานอุตสาหกรรมที่ออกแบบมาให้ทนทานต่อสภาพแวดล้อม และควบคุม กระบวนการได้อย่างมีความแน่นอนเชิงเวลา โดยทำงานเป็นรอบสแกน (scan cycle) ที่ตายตัว คือ อ่านค่าอินพุต ประมวลผลตามโปรแกรม แล้วเขียนค่าเอาต์พุต และเขียนโปรแกรมด้วยภาษาตาม มาตรฐาน IEC 61131-3 โดยภาษาที่นิยมใช้มากที่สุดสองภาษาคือ แผนภาพแลดเดอร์ (Ladder Diagram: LD) ซึ่งเขียนตรรกะเป็นขั้น (rung) ของหน้าสัมผัส (contact) และคอยล์ (coil) ในลักษณะ เดียวกับวงจรควบคุมแบบรีเลย์ จึงเป็นที่คุ้นเคยของช่างไฟฟ้า และแผนภาพบล็อกฟังก์ชัน (Function Block Diagram: FBD) ซึ่งสร้างตรรกะด้วยการลากเส้นเชื่อมบล็อกสำเร็จรูป เช่น เกตตรรกะ ตัวตั้ง เวลา และตัวนับ เข้าด้วยกัน เหมาะกับการมองการไหลของสัญญาณเป็นภาพรวม ทั้งนี้ LOGO! Soft Comfort ที่ใช้ในงานนี้รองรับทั้งสองภาษา (ตัวอย่างโปรแกรมแลดเดอร์แสดงในภาพที่ 3.6) จุดเด่น คือการให้สัญญาณและพอร์ต I/O ระดับอุตสาหกรรมจริงพร้อมการรับรองมาตรฐาน แต่ก็มีผู้คิด กับชุดเครื่องมือเฉพาะของผู้ผลิต ในงานนี้เลือกใช้พีแอลซีรุ่น Siemens LOGO! 24CE (ตระกูล LOGO! 8 เฟิร์มแวร์ V8.4) ซึ่งมีเซิร์ฟเวอร์ Modbus TCP มาให้ในตัว (รายละเอียดอยู่ในหัวข้อ 3.2 และภาคผนวก จ)

2.5 ชุดเครื่องมือโอเพนซอร์สเทียบกับชุดเครื่องมือเฉพาะผู้ผลิต

นอกเหนือจากเรื่องหน่วยประมวลผลและพอร์ต I/O แล้ว ตัวควบคุมทั้งสามประเภทยัง แตกต่างกันอย่างชัดเจนในด้านความเปิดกว้างของซอฟต์แวร์ ซึ่งเป็นประเด็นที่งานนี้ให้ความสำคัญ เป็นลำดับแรกในเชิงการเรียนการสอน โดย MCU เขียนโปรแกรมด้วย Arduino core ที่เป็นโอเพน ซอร์สร่วมกับระบบบิลด์ PlatformIO ส่วน SBC ทำงานบน Linux, Docker, ชุด Telegraf-InfluxDB- Grafana, Node-RED และโพรกเกอร์ MQTT อย่าง Mosquitto ซึ่งล้วนเป็นซอฟต์แวร์โอเพนซอร์ส ในทางตรงกันข้าม PLC เขียนโปรแกรมได้เฉพาะผ่านซอฟต์แวร์ LOGO! Soft Comfort ของ Siemens ที่เป็นซอฟต์แวร์ปิดและต้องมีใบอนุญาต ชุดซอฟต์แวร์แบบเปิดสามารถตรวจสอบโค้ด เขียนสคริปต์ ทำซ้ำได้โดยไม่มีค่าใช้จ่าย และมีชุมชนคอยสนับสนุน ในขณะที่ชุดซอฟต์แวร์เฉพาะ ของผู้ผลิตแลกมาด้วยการสนับสนุนและการรับรองจากผู้ผลิต แต่ก็ผูกผู้ใช้ไว้กับผู้จำหน่ายเพียงราย เดียว ความแตกต่างนี้ทำให้นักศึกษาได้สัมผัสประสบการณ์การผูกติดกับผู้ผลิต (vendor lock-in) ด้วยตนเองโดยตรง แทนที่จะเป็นเพียงแนวคิดเชิงนามธรรม และเป็นเหตุผลที่แพลตฟอร์มนี้จึงใจนำ ทั้งสองโลกมาผสมผสานเข้าด้วยกัน

2.6 ชุดแดชบอร์ดข้อมูลอนุกรมเวลา (TIG)

ข้อมูลโทรมาตร (telemetry) ในงานนี้ถูกจัดเก็บและแสดงผลด้วยชุดซอฟต์แวร์ TIG ซึ่งเป็นชุดโอเพนซอร์สที่ได้รับความนิยมอย่างแพร่หลาย โดย Telegraf ทำหน้าที่เป็นเอเจนต์เก็บรวบรวมข้อมูลแบบปลั๊กอิน อินพุต mqtt_consumer จะสมัครรับข้อความจากโบรกเกอร์ และเอาต์พุต influxdb_v2 จะเขียนข้อมูลลงฐานข้อมูล ขณะที่อินพุตอื่น ๆ ช่วยเก็บข้อมูลสถานะของเครื่องโฮสต์เพิ่มเติม [6] ส่วน InfluxDB เป็นฐานข้อมูลอนุกรมเวลา (time-series database) ที่เก็บจุดข้อมูลไว้ในบั๊กเก็ต (bucket) และสืบค้นด้วยภาษา Flux [7] และ Grafana ทำหน้าที่แสดงผลแดชบอร์ดจากการสืบค้นเหล่านั้น พร้อมรองรับการเข้าดูแบบอ่านอย่างเดียวโดยไม่ต้องระบุตัวตนสำหรับจอแสดงผลแบบคีออสก์ (kiosk) [8] ทั้งสามองค์ประกอบเป็นอิมเมจ Docker และจัดการรวมกันด้วย Docker Compose

2.7 โบรกเกอร์ MQTT (Mosquitto)

โบรกเกอร์ Mosquitto เชื่อมต่ออุปกรณ์ชั้นขอบเข้ากับชุดแดชบอร์ดผ่าน MQTT ซึ่งเป็นโพรโทคอลแบบผู้เผยแพร่-ผู้สมัครรับข้อความที่อธิบายไว้ในหัวข้อ 2.3.3 อุปกรณ์ชั้นขอบทำหน้าที่เป็นผู้เผยแพร่ข้อความ (publisher) ส่งข้อมูลโทรมาตรไปยังหัวข้อ (topic) บนโบรกเกอร์ และอินพุต mqtt_consumer ของ Telegraf ทำหน้าที่เป็นผู้สมัครรับข้อความ (subscriber) ในหัวข้อเหล่านั้นเพื่อนำข้อมูลเข้า เช่นเดียวกับองค์ประกอบของชุด TIG นั้น Mosquitto มีน้ำหนักเบา ใช้งานแพร่หลาย และถูกเผยแพร่เป็นอิมเมจ Docker ที่จัดการด้วย Docker Compose

2.8 Node-RED

Node-RED (โนนด-เรด) เป็นเครื่องมือสำหรับพัฒนาโปรแกรมแบบโฟลว์ (flow-based programming) ที่ทำงานบนแพลตฟอร์ม Node.js และใช้งานผ่านหน้าแก้ไขโฟลว์บนเว็บเบราว์เซอร์ แทนที่จะเขียนโค้ดที่ละบรรทัด ผู้ใช้จะสร้างตรรกะการทำงานด้วยการนำ “โนนด” (node) มาวางบนพื้นที่ทำงานแล้วลาก “สาย” (wire) เชื่อมต่อกันเป็นโฟลว์ โดยข้อมูล (message) จะไหลจากโนนดหนึ่งไปยังอีกโนนดหนึ่งตามเส้นที่เชื่อมต่อไว้ Node-RED มีโนนดสำเร็จรูปให้เลือกใช้จำนวนมาก ทั้งโนนดอินพุตสำหรับกระตุ้นการทำงานตามเวลา (inject) โนนดฟังก์ชันสำหรับเขียนตรรกะด้วยภาษา JavaScript และโนนดเสริมจากชุมชนสำหรับเชื่อมต่อโพรโทคอลต่าง ๆ เช่น MQTT และ Modbus จุดเด่นของ Node-RED คือช่วยให้สร้างต้นแบบและเชื่อมประสานอุปกรณ์เข้าด้วยกันได้อย่างรวดเร็ว มองเห็นการไหลของข้อมูลได้ชัดเจน และดีบั๊กได้ง่าย จึงเป็นเครื่องมือที่นิยมใช้กันแพร่หลายในงานไอโอที ในงานนี้ Node-RED ทำงานอยู่บน Raspberry Pi ในฝั่งไอที และถูกใช้

พัฒนาโพลีสำหรับควบคุมและอ่านค่าจากเกตเวย์ ESP32-S3 ผ่าน Modbus TCP ซึ่งจะกล่าวถึงรายละเอียดในหัวข้อ 3.7

2.9 งานวิจัยที่เกี่ยวข้องและช่องว่างงานวิจัย

งานวิจัยก่อนหน้านี้มีทั้งการสร้างเกตเวย์ Modbus บน ESP32 ชุดทดสอบสำหรับเฝ้าติดตามงานไอโอที และงานสำรวจโพรโทคอลด้านไอโอทีที่จำนวนมาก [1]-[5] สิ่งที่วรรณกรรมเหล่านี้พบตรงกันคือ มีเครื่องมือโอเพนซอร์สกลุ่มเล็ก ๆ ที่ปรากฏเข้าไปป้อนข้อมูลไอโอทีที่ให้แก่โบรกเกอร์ MQTT ทำหน้าที่เป็นโครงข่ายรับส่งข้อความ Node-RED เป็นชั้นบูรณาการแบบโพลี และชุด TIG (Telegraf, InfluxDB, Grafana) สำหรับเก็บรวบรวม จัดเก็บ และแสดงผลข้อมูล [6]-[8] งานนี้จึงเลือกใช้ชุดเทคโนโลยีที่เป็นมาตรฐานเหล่านี้แทนการคิดค้นองค์ประกอบขึ้นใหม่ เพื่อให้แพลตฟอร์มเป็นที่คุ้นเคยของผู้ใช้งานและทำซ้ำได้ง่าย อย่างไรก็ตามยังมีช่องว่างเหลืออยู่สองประการ ประการแรก ชุดทดสอบที่มีอยู่มักขาดตัวควบคุมเพียงประเภทเดียว ไม่ได้นำ MCU, SBC และ PLC มาบูรณาการเพื่อเปรียบเทียบบทบาทของแต่ละประเภท ประการที่สอง แม้ไลบรารี Modbus แบบโอเพนซอร์สสำหรับ ESP32 จะมีให้เลือกมาก แต่ยังไม่มียุทธศาสตร์เชิงประจักษ์บนฮาร์ดแวร์ชุดเดียวกันที่ใช้เป็นแนวทางการเลือกได้ ปริมาณงานนี้จึงปิดช่องว่างทั้งสองประการด้วยการบูรณาการตัวควบคุมทั้งสามประเภทบนชุดทดลองต้นทุนต่ำชุดเดียว ทั้งนี้ยังทดสอบสมรรถนะของไลบรารีเกตเวย์ Modbus แบบโอเพนซอร์สจำนวนแปดตัวเพื่อคัดเลือกไลบรารีที่นำไปใช้งาน

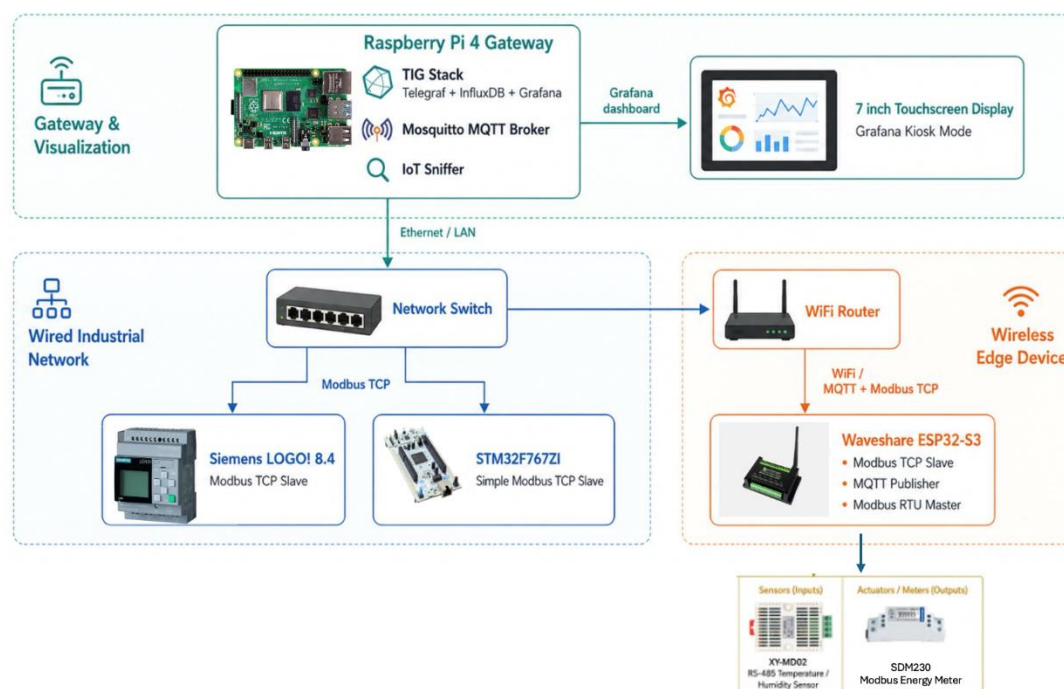
บทที่ 3

การออกแบบและพัฒนาระบบ

บทนี้นำเสนอการออกแบบและพัฒนาแพลตฟอร์ม โดยเริ่มจากภาพรวมของสถาปัตยกรรม และการกำหนดเส้นทางการไหลของข้อมูล จากนั้นอธิบายเหตุผลในการเลือกฮาร์ดแวร์ทั้งสามประเภท และปิดท้ายด้วยรายละเอียดของซอฟต์แวร์ในแต่ละชั้น ทั้งการเขียนโปรแกรม PLC เฟิร์มแวร์ของเอจเกตเวย์บน ESP32-S3 ชุดซอฟต์แวร์ฝั่งไอทีแบบคอนเทนเนอร์ และเครื่องมือ IoT-Sniffer

3.1 สถาปัตยกรรมโดยรวม

แพลตฟอร์มนี้บูรณาการตัวควบคุมทั้งสามประเภทเข้าไว้ด้วยกันบนเครือข่ายท้องถิ่นเดียว โดยมีเครื่องมือวัดหน้างานคือ เซนเซอร์อุณหภูมิ และความชื้น XY-MD02 กับมิเตอร์ไฟฟ้าเฟสเดียว Easton SDM230 ซึ่งทั้งคู่ทำหน้าที่เป็นสเลฟ Modbus RTU บนบัส RS-485 ส่วนพีแอลซี Siemens LOGO! 24CE ทำหน้าที่ควบคุมที่มีความแน่นอนเชิงเวลาผ่านพอร์ต I/O ระดับอุตสาหกรรม ในขณะเดียวกัน เอจเกตเวย์ ESP32-S3 ทำหน้าที่แปลงโปรโตคอล และ Raspberry Pi 4 ทำหน้าที่รองรับโปรแกรม MQTT และชั้นข้อมูล TIG ทั้งนี้การออกแบบนี้เปิดเส้นทางการไหลของข้อมูลไว้สองเส้นทาง เส้นทาง A คือการสื่อสารด้วย Modbus TCP โดยตรง ซึ่งโหนดใด ๆ ในเครือข่าย LAN รวมถึงพีแอลซีที่ทำหน้าที่เป็นมาสเตอร์ สามารถเข้าไปสำรวจเซิร์ฟเวอร์ Modbus TCP ได้ ส่วนเส้นทาง B คือไปป์ไลน์ของข้อมูลโทรมาตร โดย ESP32-S3 จะสำรวจเซนเซอร์ RTU แล้วเผยแพร่ข้อมูลในรูปแบบ JSON ผ่าน MQTT จากนั้น Telegraf จะนำข้อมูลเข้าสู่ InfluxDB เพื่อนำไปแสดงผลใน Grafana ดังแสดงในภาพที่ 3.1



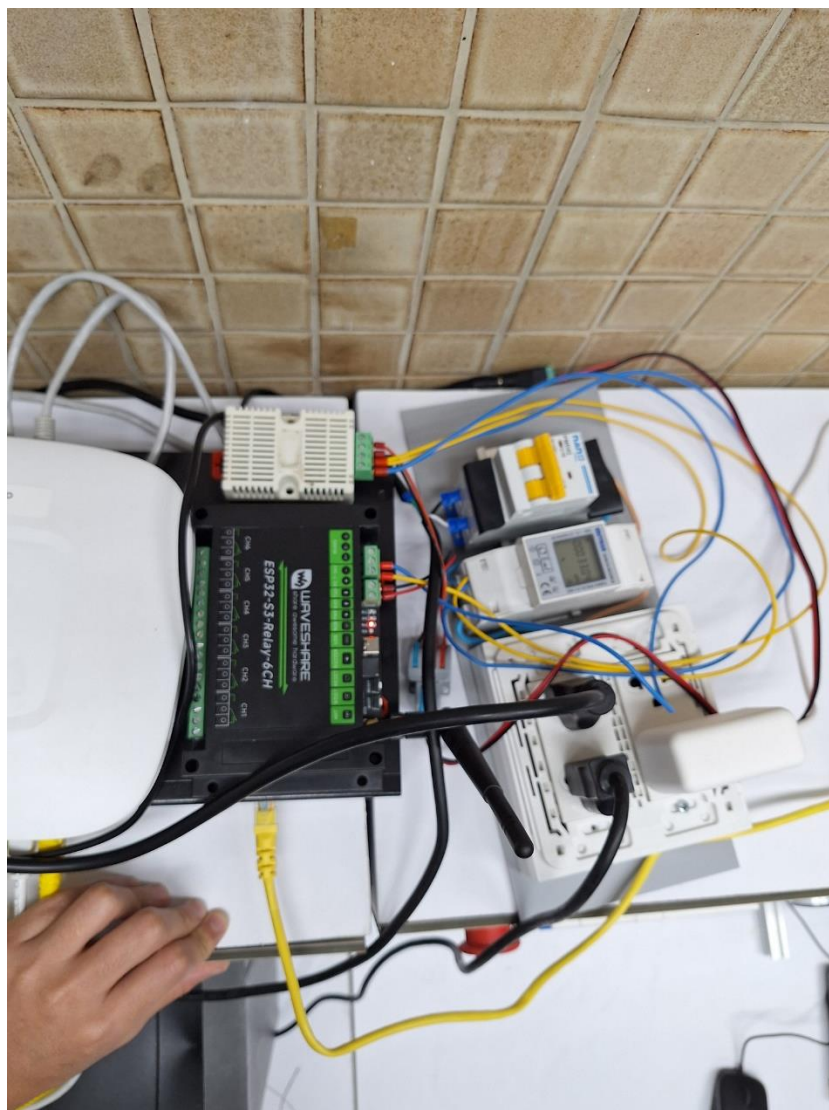
ภาพที่ 3.1 สถาปัตยกรรมโดยรวมของแพลตฟอร์ม: เครื่องข่ายอุตสาหกรรมแบบมีสาย เกตเวย์ Raspberry Pi 4 และอุปกรณ์ขอบไร้สาย ESP32-S3 (สร้างด้วย AI และปรับแก้บางส่วน)

3.2 การเลือกฮาร์ดแวร์ MCU, SBC และ PLC

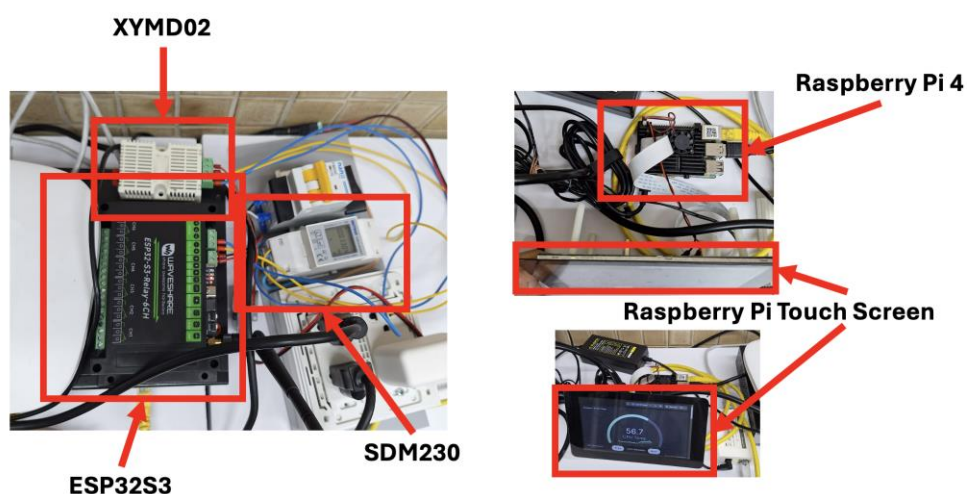
การเลือกฮาร์ดแวร์ยึดหลักให้ครอบคลุมตัวควบคุมครบทั้งสามประเภทด้วยต้นทุนต่ำที่สุด โดยยังคงใช้สัญญาณอุตสาหกรรมจริง สำหรับ MCU เลือกใช้บอร์ด Waveshare ESP32-S3-Relay-6CH เนื่องจากมีตัวรับส่งสัญญาณ RS-485 มาในตัว มีเอาต์พุตรีเลย์แบบ SPDT หกช่อง มี PSRAM และรองรับ PlatformIO โดยมีต้นทุนบอร์ดประมาณ 1,895 บาท ด้าน SBC เลือกใช้ Raspberry Pi 4 (2 GB) เพราะมีสภาพแวดล้อม Linux รองรับ Docker และมีพอร์ตอีเทอร์เน็ต ส่วน PLC เลือกใช้ Siemens LOGO! 24CE (รหัสสินค้า 6ED1052-1CC08-0BA2) ซึ่งเป็นอุปกรณ์ขนาด 24 V DC ที่มีเอาต์พุตทรานซิสเตอร์สี่ช่องและเซอร์ฟเวอร์ Modbus TCP ในตัว โดยเลือกเพราะให้พอร์ต I/O ระดับอุตสาหกรรมจริงและผ่านการรับรอง สำหรับเครื่องมือวัดหน้างานเลือกใช้ XY-MD02 และ SDM230 เพราะทั้งคู่เป็นสเลฟ Modbus RTU ราคาประหยัดที่ให้ค่าการวัดแบบเดียวกับงานอุตสาหกรรมจริง รวมถึงการส่งค่าเป็นคูรีจิสเตอร์ทศนิยม IEEE-754 ของมิเตอร์ไฟฟ้า ดังแสดงในภาพที่ 3.2

บอร์ด Waveshare ESP32-S3-Relay-6CH ที่เลือกใช้เป็น MCU ในงานนี้ไม่ใช่บอร์ดพัฒนา ESP32 ทั่วไป แต่เป็นบอร์ดรีเลย์เครือข่ายระดับอุตสาหกรรม (industrial-grade network relay) ที่

รวมเอาโมดูล Espressif ESP32-S3-WROOM-1 ซึ่งใช้ตัวประมวลผล Xtensa LX7 แบบสองแกน ความเร็วสูงสุด 240 MHz พร้อม Wi-Fi และ Bluetooth LE ไว้กับวงจรควบคุมกำลังไฟฟ้าบนแผ่นวงจรเดียวกัน จุดเด่นที่ทำให้บอร์ดนี้เหมาะกับโครงการคือ ความสามารถที่มีมาให้ในตัวสามอย่าง ประการแรก มีรีเลย์จำนวนหกช่อง (หน้าสัมผัสแบบ 1NO 1NC) พิกัดหน้าสัมผัส $\leq 10\text{ A}$ ที่ 250 V AC หรือ $\leq 10\text{ A}$ ที่ 30 V DC สำหรับตัดต่อโหลดไฟฟ้าจริง ประการที่สอง มีอินเทอร์เฟซ RS-485 แบบแยกทางไฟฟ้า (isolated RS485) มาในตัว ทำให้เชื่อมต่อกับอุปกรณ์และเซนเซอร์ Modbus บนบัส RS-485 ได้ทันที โดยไม่ต้องต่อโมดูลตัวรับส่งสัญญาณเพิ่มจากภายนอก และประการที่สาม มีวงจรแยกทางไฟฟ้าครบชุด ทั้งการแยกด้วยออปโตคัปเปิลเลอร์ (optocoupler isolation) การแยกสัญญาณดิจิทัล (digital isolation) และการแยกแหล่งจ่ายไฟ (power isolation) ในตัว ซึ่งช่วยป้องกันไม่ให้สัญญาณรบกวนหรือแรงดันสูงจากฝั่งโหลดย้อนกลับมาทำอันตรายต่อชิปควบคุม นอกจากนี้บอร์ดยังมีบัสเซอร์ ไฟแสดงสถานะแบบ RGB และไฟแสดงการรับส่งของ RS-485 สำหรับสังเกตการทำงาน พร้อมขั้วต่อแบบ Pico ที่รองรับการต่อ HAT เพิ่มเติม เช่น RTC, CAN หรือ LoRa เพื่อขยายความสามารถในอนาคต ด้วยการรวมทุกอย่างไว้บนบอร์ดเดียวเช่นนี้ ESP32-S3-Relay-6CH จึงทำหน้าที่เป็นเอดจ์เกตเวย์ที่พร้อมเชื่อมทั้งสัญญาณ RS-485 ฝั่งไอทีและเครือข่าย Wi-Fi ฝั่งไอทีได้ในอุปกรณ์ชิ้นเดียว โดยขาที่ใช้ควบคุมรีเลย์คือ GP1, GP2, GP41, GP42, GP45 และ GP46 ส่วนอินเทอร์เฟซ RS-485 ใช้ขา GP17 (TX) และ GP18 (RX) และพัฒนาเฟิร์มแวร์ด้วย Arduino core ผ่าน PlatformIO ภาพรวมการติดตั้งฮาร์ดแวร์ทั้งหมดของแพลตฟอร์ม ทั้งฝั่งเกตเวย์และฝั่งชั้นไอที แสดงในภาพที่ 3.3



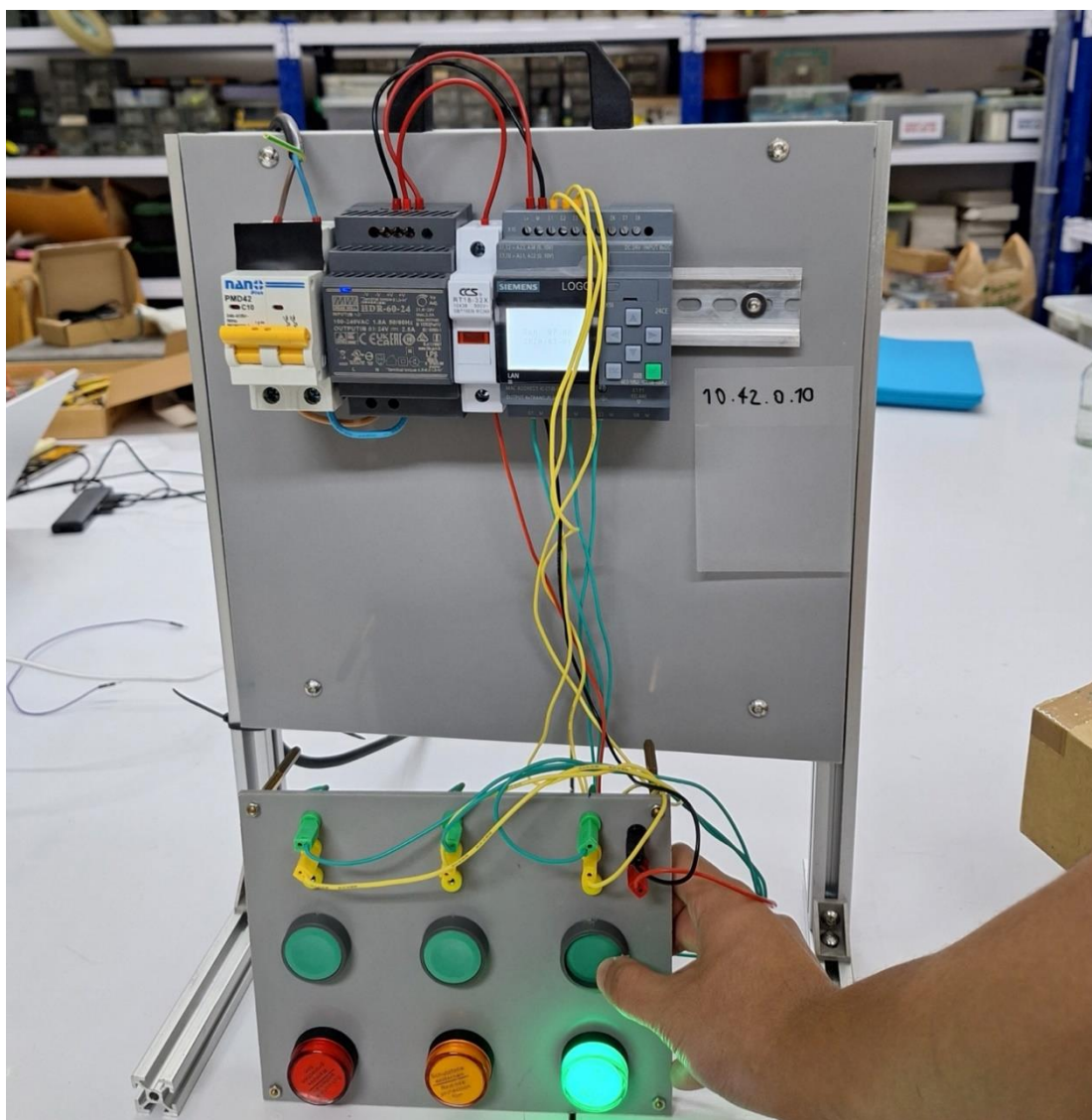
ภาพที่ 3.2 ฮาร์ดแวร์จริงของแพลตฟอร์ม: เกตเวย์ ESP32-S3-Relay-6CH, XY-MD02 และ SDM230 พร้อมการเดินสาย RS-485



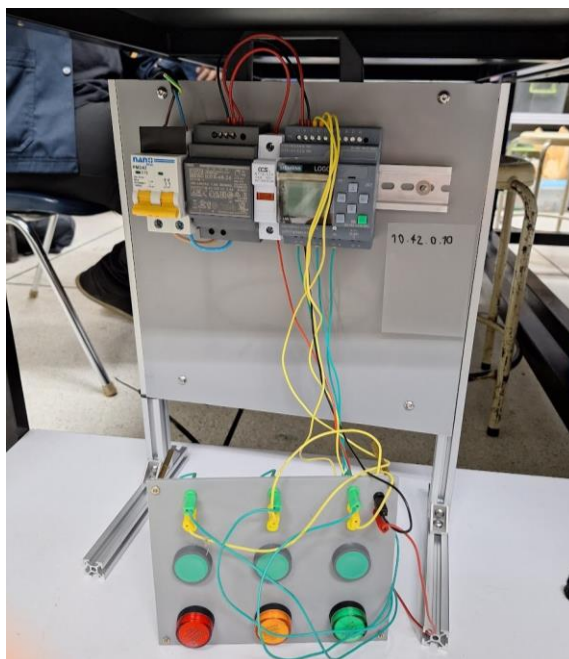
ภาพที่ 3.3 การติดตั้งฮาร์ดแวร์ของแพลตฟอร์ม: เกตเวย์ ESP32-S3-Relay-6CH ร่วมกับ XY-MD02 และ SDM230 (ซ้าย) และ Raspberry Pi 4 พร้อมจอสัมผัส (ขวา)

3.3 ซอฟต์แวร์ชั้นไอที: การเขียนโปรแกรม PLC ด้วย LOGO! Soft Comfort

LOGO! 24CE เขียนโปรแกรมได้ผ่านซอฟต์แวร์ LOGO! Soft Comfort ซึ่งเป็นสภาพแวดล้อมตามมาตรฐาน IEC 61131-3 เฉพาะของผู้ผลิตเท่านั้น จึงสะท้อนประเด็นเรื่องชุดเครื่องมือแบบปิดตามที่กล่าวไว้ในหัวข้อ 2.5 ตรรกะการควบคุมถูกเขียนขึ้นในรูปแบบแผนภาพบล็อกฟังก์ชันหรือแผนภาพแลดเดอร์ แล้วดาวน์โหลดลงอุปกรณ์ผ่านอีเทอร์เน็ต โปรแกรมตัวอย่างหนึ่งชุดใช้การสั่งสตาร์ท/หยุดแบบล็อกตัวเอง (self-latching) พร้อมตัวนับเวลาทำงานที่เก็บไว้ใน V-memory ทั้งนี้เซิร์ฟเวอร์ Modbus TCP ของพีแอลซีถูกเปิดใช้งานภายในเครื่องมือเดียวกัน และจุดสัญญาณดิจิทัลกับแอนะล็อกของพีแอลซีถูกแมปไปยังที่อยู่ Modbus (ทั้ง coils และ word ใน V-memory) เพื่อให้โฮสต์ใด ๆ ในเครือข่าย LAN อ่านหรือเขียนค่าได้ นอกจากนี้ LOGO! ยังสามารถทำหน้าที่เป็นมาสเตอร์ Modbus TCP โดยอ่านค่า holding registers ของ ESP32-S3 เข้ามาเก็บไว้ใน V-memory ของตัวเอง แสดงให้เห็นว่าเมื่อทั้งสองฝั่งสื่อสารด้วย Modbus TCP ร่วมกันแล้ว ทิศทางการไหลของข้อมูลเป็นเพียงเรื่องของการตั้งค่า ไม่ใช่ข้อจำกัดในเชิงสถาปัตยกรรมดังแสดงในภาพที่ 3.4 และภาพที่ 3.5

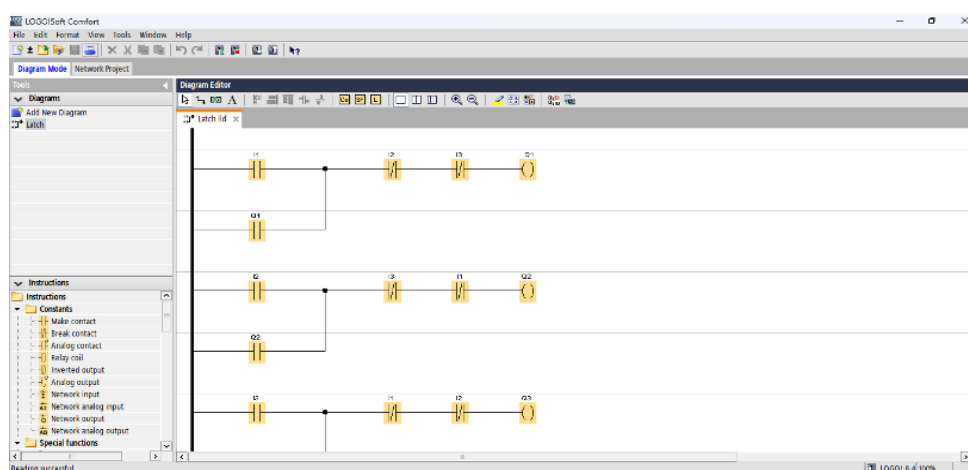


ภาพที่ 3.4 การเดินสายภายในแผงควบคุม Siemens LOGO! 24CE



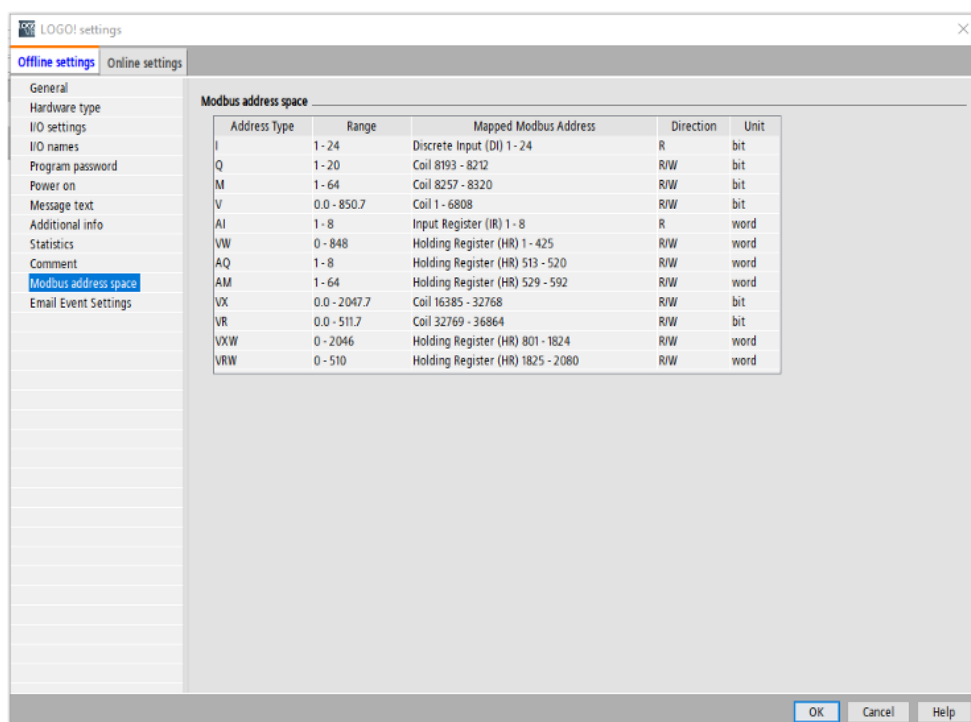
ภาพที่ 3.5 แผงสายิตพีแอลซีที่ประกอบเสร็จ พร้อมสวิตช์ปุ่มกดและไฟแสดงสถานะ

ตรรกะควบคุมตัวอย่างของแผงสายิตเขียนขึ้นในรูปแบบแผนภาพแลคเตอร์ใน LOGO! Soft Comfort ดังแสดงในภาพที่ 3.6 โดยเอาต์พุต Q1-Q3 แต่ละช่องใช้วงจรล็อกตัวเอง (self-latching) คือหน้าสัมผัสช่วยของเอาต์พุตต่อขนานกับปุ่มสตาร์ท ทำให้เอาต์พุตคงสถานะติดไว้หลังปล่อยปุ่ม และดับลงเมื่อกดปุ่มหยุดที่ต่อเป็นหน้าสัมผัสปกติปิด (normally closed)



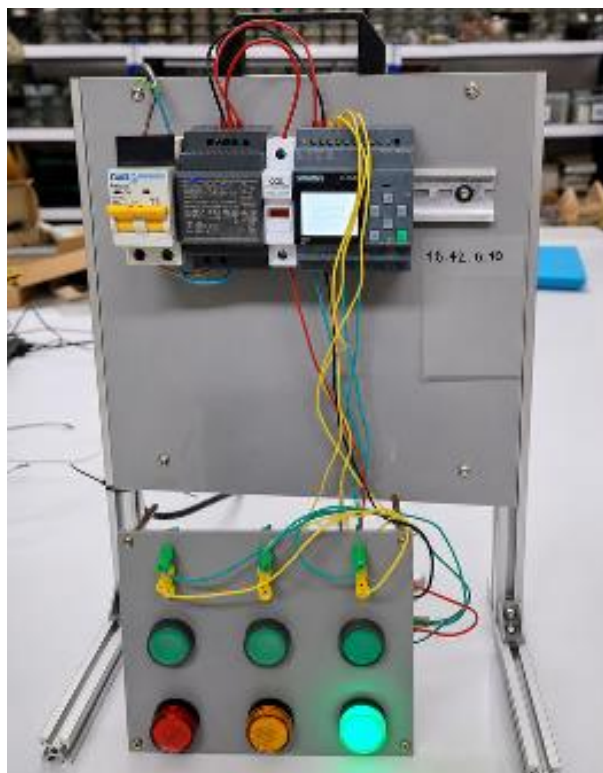
ภาพที่ 3.6 ตัวอย่างโปรแกรมแลคเตอร์แบบล็อกตัวเอง (self-latching) สำหรับควบคุมเอาต์พุต Q1-Q3 ใน LOGO! Soft Comfort

การเข้าถึงพีแอลซีจากภายนอกผ่าน Modbus TCP อาศัยการแมปพื้นที่ที่อยู่ภายในของ LOGO! ไปยังที่อยู่ Modbus มาตรฐาน ซึ่งกำหนดไว้ในหน้าต่าง Modbus address space ของ LOGO! Soft Comfort ดังแสดงในภาพที่ 3.7 โดยเอาต์พุตดิจิทัล Q ถูกแมปเป็น coil หมายเลข 8193-8212 อินพุต I เป็น discrete input หมายเลข 1-24 และ word ใน V-memory เป็น holding registers



ภาพที่ 3.7 หน้าต่าง Modbus address space ของ LOGO! Soft Comfort แสดงการแมปเอาต์พุต Q เป็น coil 8193-8212 และ V-memory เป็น holding registers

เพื่อยืนยันการทำงาน สามารถสั่งเปิด-ปิดเอาต์พุตของพีแอลซีจากโฮสต์ใดก็ได้ในเครือข่ายด้วยเครื่องมือ mbpoll เช่น คำสั่ง mbpoll -m tcp -a 1 -t 0 -r 8193 10.42.0.10 1 เขียนค่า 1 ไปยัง coil 8193 เพื่อสั่งเปิดเอาต์พุต Q1 ของพีแอลซี และคำสั่งเดียวกันที่ลงท้ายด้วยค่า 0 ใช้สั่งปิดเอาต์พุต ผลการทดสอบแสดงในภาพที่ 3.8 ซึ่งไฟแสดงสถานะบนแผงสาธิตติดสว่างตามคำสั่งที่ส่งเข้ามาทางเครือข่าย



ภาพที่ 3.8 ผลการสั่งเปิดเอาต์พุตของรีเลย์ผ่าน Modbus TCP ด้วยคำสั่ง mbpoll:
ไฟแสดงสถานะบนแผงสวิตช์ติดสว่างเมื่อเขียนค่า 1 ไปยัง coil 8193

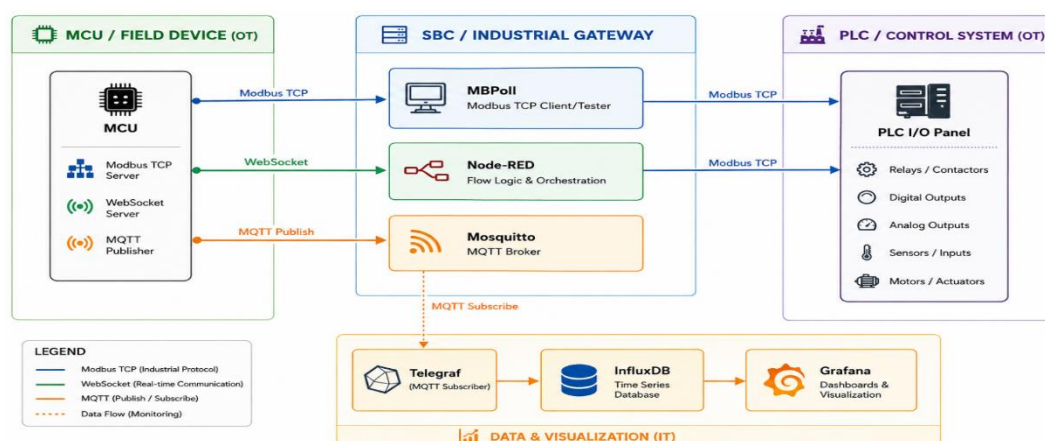
3.4 เฟิร์มแวร์เอดจ์เกตเวย์: Modbus บน ESP32-S3

เฟิร์มแวร์ของเกตเวย์เป็นแอปพลิเคชันบน Arduino core ที่พัฒนาด้วย PlatformIO และทำงานสามบทบาทพร้อมกันภายในรูปแบบรวมมือกันที่ไม่บล็อกการทำงาน (non-blocking) โดยวนทำงานทุก ๆ สองวินาที และไม่เรียกใช้คำสั่ง delay() เลย บทบาทแรกคือ การเป็นมาสเตอร์ Modbus RTU ที่เขียนขึ้นเอง ทำหน้าที่สำรวจ XY-MD02 และ SDM230 ผ่าน RS-485 โดยประกอบเฟรมคำขอพร้อมค่า CRC-16 ตรวจจับการตอบกลับแบบข้อยกเว้น (exception) และบันทึกรหัสสถานะรวมของแต่ละสเลฟ บทบาทที่สองคือ การเป็นตัวเผยแพร่ MQTT (ใช้ไลบรารี 256dpi/MQTT) ที่ส่งค่าที่อ่านได้ในรูปแบบเอกสาร JSON ไปยังโบรกเกอร์ด้วย QoS 1 เมื่อเกิดข้อผิดพลาดที่ฝั่งอุปกรณ์ ระบบจะละฟิลด์ของค่าที่ได้รับผลกระทบและส่งเฉพาะรหัสสถานะ เพื่อไม่ให้ปลายทางได้รับค่าที่ไม่ถูกต้อง บทบาทที่สามคือ การเป็นสเลฟ Modbus TCP (ใช้ไลบรารี emelianov/modbus-esp8266) ที่ให้บริการค่าที่อ่านได้ชุดเดียวกันบนพอร์ต 502 ในรูปของ holding registers ตั้งแต่ HR0 ถึง HR20 ประกอบด้วยค่าจำนวนเต็มที่ปรับสเกลแล้วสำหรับอุณหภูมิและความชื้น ตัวนับสถานะและตัวนับการสำรวจ ตลอดจนคู่ word สูง/ต่ำสำหรับค่าทศนิยม IEEE-754

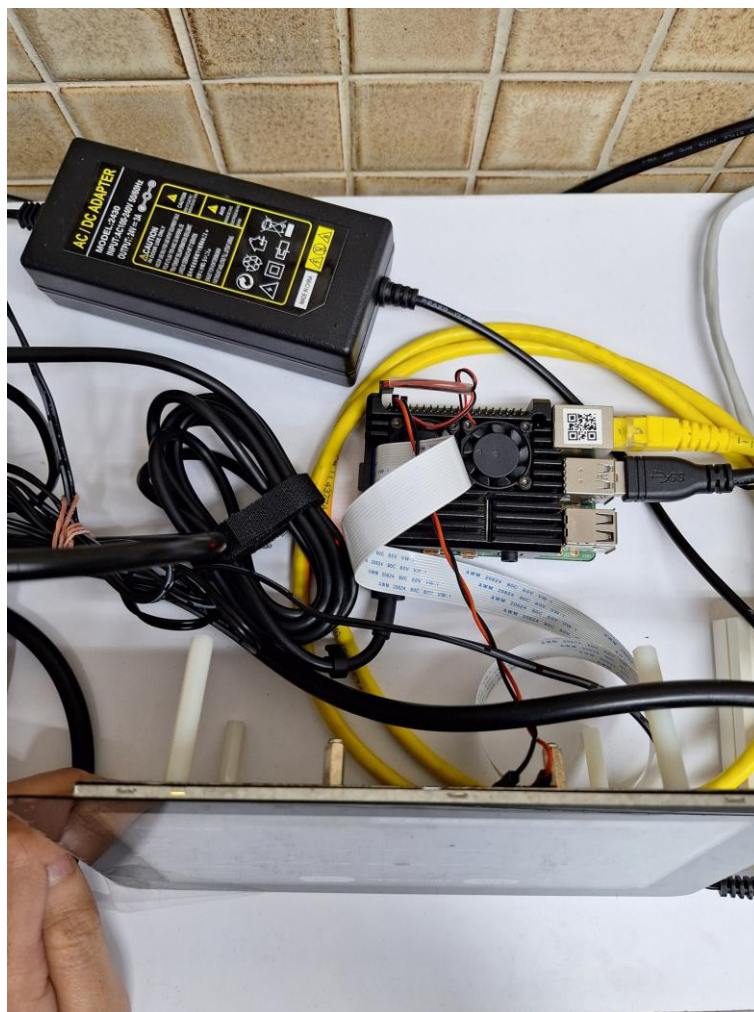
ทั้งแปดค่าของ SDM230 ทั้งสามบทบาทนี้รวมถึงวอตช์ด็อก (watchdog) ทำงานร่วมกันแบบไม่ให้บทบาทใดขัดขวางอีกบทบาทหนึ่ง ผลการบีลัดใช้พื้นที่แฟลช 946 KB จากพาร์ทิชันแอปพลิเคชัน 1,310 KB (คิดเป็นร้อยละ 72.2) และใช้แรม 48.5 KB จาก 327.6 KB (ร้อยละ 14.8) ขึ้นันว่าเกตเวย์นี้เหมาะสมกับ MCU ต้นทุนต่ำโดยยังมีทรัพยากรเหลืออยู่มาก

3.5 ขอฟต์แวร์ชั้นไอที: MQTT และชุด TIG แบบคอนเทนเนอร์

Raspberry Pi ทำหน้าที่โฮสต์ชั้นข้อมูลในรูปของบริการ Docker Compose โดยระบุแท็กของอิมเมจไว้อย่างชัดเจน และกำหนดค่าความลับผ่านตัวแปรสภาพแวดล้อม Mosquitto 2.0 เป็นโบรกเกอร์ MQTT รับฟังบนพอร์ต 1883 และพอร์ต 9001 สำหรับ WebSocket พร้อมเปิดการหมุนเวียนไฟล์ล็อกของ Docker ส่วน InfluxDB 2.7 จัดเก็บข้อมูลโทรมาตรไว้ในบักเก็ต sensors ภายใต้องค์กร weatherstation ทั้งนี้ Telegraf 1.28 เชื่อมสององค์ประกอบนี้เข้าด้วยกัน กล่าวคือ อินพุต mqtt_consumer สัมผัสรับข้อความด้วย QoS 1 แล้วแปลงข้อมูล JSON จากนั้นเอาต์พุต influxdb_v2 เขียนลงฐานข้อมูล และยังมีอินพุตเสริมเก็บสถานะของ Raspberry Pi ทั้งซีพียู หน่วยความจำ ดิสก์ อุณหภูมิ และการเชื่อมต่อเครือข่าย จากนั้น Grafana แสดงผลแดชบอร์ดค่าสภาพอากาศ ค่าทางไฟฟ้า สถานะโฮสต์ และเครือข่าย พร้อมบทบาทผู้ชมแบบไม่ระบุตัวตนสำหรับจอแสดงผลแบบคีย์บอร์ด เนื่องจากทุกองค์ประกอบเป็นโอเพนซอร์สและประกาศไว้ในไฟล์ Compose เพียงไฟล์เดียว นักศึกษาคนอื่นจึงติดตั้งชั้นไอทีทั้งหมดซ้ำได้บนฮาร์ดแวร์มาตรฐานทั่วไป ดังแสดงในภาพที่ 3.9 และภาพที่ 3.10



ภาพที่ 3.9 เส้นทางการไหลของข้อมูลจากอุปกรณ์หน้างานผ่านเกตเวย์สู่ชั้นไอที (MBPoll, Node-RED, Mosquitto, Telegraf, InfluxDB และ Grafana) (สร้างด้วย AI และปรับแก้บางส่วน)

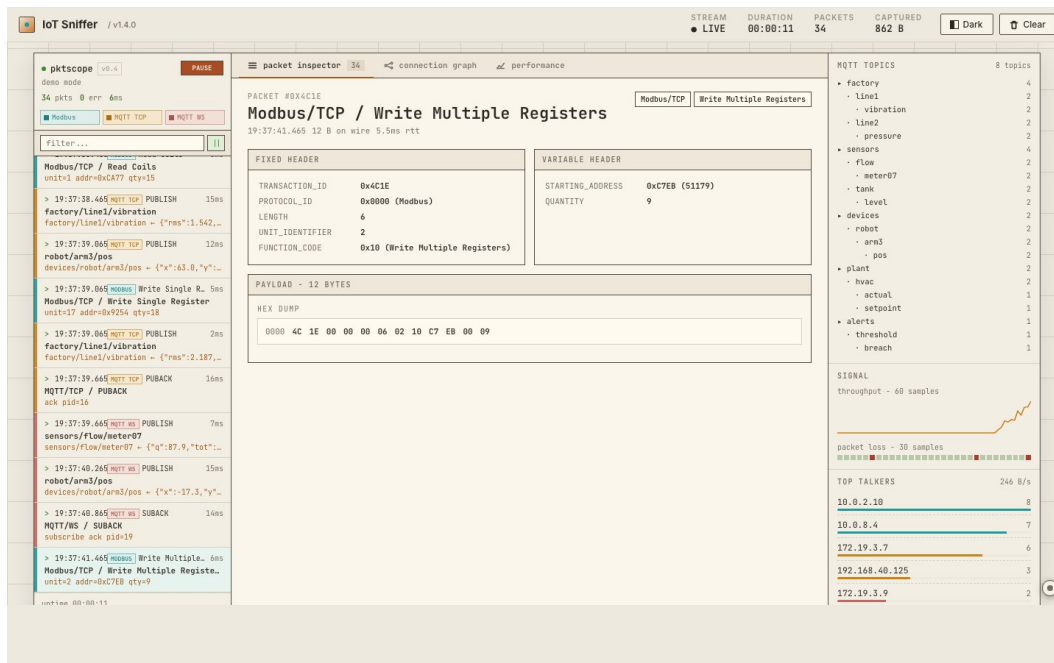


ภาพที่ 3.10 Raspberry Pi 4 ที่ติดตั้งใช้งานจริงพร้อมแหล่งจ่ายไฟและสายอีเทอร์เน็ต

3.6 เครื่องมือ IoT-Sniffer

เพื่อให้สังเกตพฤติกรรมในระดับสายสัญญาณได้ งานนี้ได้พัฒนาเครื่องมือดักจับข้อมูลแบบเชิงรับ (passive) ขึ้นด้วยภาษา Python เครื่องมือนี้ดักจับการรับส่งข้อมูลบนอินเทอร์เน็ตเฟสเครือข่ายด้วย AsyncSniffer ของไลบรารี scapy แล้วประกอบข้อมูลของแต่ละโฟลว์ TCP กลับคืนจากนั้นถอดรหัสเฟรมด้วยตัวแยกวิเคราะห์ (parser) ที่เขียนด้วย Python ล้วน กล่าวคือ Modbus TCP ระบุได้จากพอร์ต 502 และค่าหมายเลขโปรโตคอลศูนย์ในส่วนหัว MBAP ส่วน MQTT ระบุได้จากโครงสร้างแพ็กเก็ตควบคุมบนพอร์ต 1883 หรือผ่าน WebSocket หลังถอดเฟรมตาม RFC 6455 นอกจากนี้เครื่องมือยังคำนวณตัวชี้วัดแบบเรียลไทม์ ทั้งอัตราการส่งข้อมูลและเวลาแฝงของคำขอที่ระดับ p50/p95/p99 บันทึกเฟรมลง SQLite ในโหมด write-ahead-log และส่งเฟรมที่ถอดรหัสแล้วพร้อมสรุปตัวชี้วัดหนึ่งครั้งต่อวินาทีไปยังแดชบอร์ดบนเบราว์เซอร์ผ่าน WebSocket ระหว่างการ

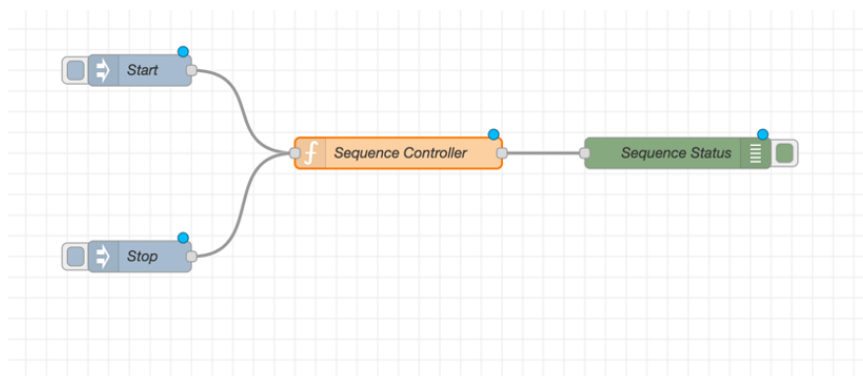
พัฒนา เครื่องมือนี้ใช้ตรวจสอบลำดับไบนารีและพฤติกรรมของโปรโตคอลบนสายสัญญาณ และยังช่วยให้ผู้เรียนเห็นทุกเฟรมของ Modbus และ MQTT ที่แพลตฟอร์มแลกเปลี่ยนกันแบบเรียลไทม์ในรูปแบบที่อ่านเข้าใจได้ ดังแสดงในภาพที่ 3.11



ภาพที่ 3.11 ส่วนต่อประสานผู้ใช้ของเครื่องมือ IoT-Sniffer ขณะถอดรหัสคำขอ Modbus TCP Write Multiple Registers

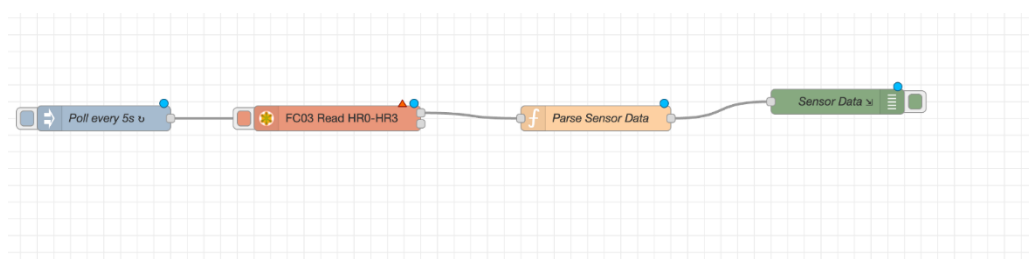
3.7 การพัฒนาฟลัวร์อัตโนมัติด้วย Node-RED

นอกจากการรับส่งข้อมูลโทรมาตรผ่าน MQTT แล้ว งานนี้ยังใช้ Node-RED ที่ทำงานบน Raspberry Pi พัฒนาฟลัวร์ขึ้นสองชุดเพื่อสาธิตการควบคุมและการอ่านค่าจากเกตเวย์ ESP32-S3 โดยตรงผ่านโปรโตคอล Modbus TCP ฟลัวร์ชุดแรกทำหน้าที่เป็นตัวควบคุมลำดับการทำงานของเอาต์พุต (LED sequencer) โดยเมื่อผู้ใช้งานกดปุ่ม Start โหนด Sequence Controller จะสั่งเปิดเอาต์พุต IO1, IO2 และ IO3 ให้ทำงานทีละช่องเรียงตามลำดับ ในทางกลับกัน เมื่อกดปุ่ม Stop ก็จะหยุดลำดับการทำงานพร้อมคืนสถานะของเอาต์พุตทั้งหมด ทั้งนี้มีโหนด Sequence Status คอยรายงานสถานะปัจจุบันของลำดับ ดังแสดงในภาพที่ 3.12



ภาพที่ 3.12 โฟลว์ Node-RED สำหรับควบคุมลำดับการเปิดเอาต์พุต IO1-IO3 (LED sequencer) ด้วยปุ่ม Start/Stop

โฟลว์ชุดที่สองทำหน้าที่อ่านค่าจากเกตเวย์เป็นระยะ โดยใช้โหนด inject กระตุ้นการทำงาน ทุก ๆ 5 วินาที เพื่อส่งคำสั่ง Modbus TCP FC03 อ่านค่า holding registers ตั้งแต่ HR0 ถึง HR3 จาก ESP32-S3 จากนั้น โหนดฟังก์ชัน Parse Sensor Data จะแปลงค่าดิบที่อ่านได้ให้อยู่ในรูปแบบที่พร้อมใช้งาน แล้วส่งต่อไปยังโหนดปลายทางเพื่อแสดงผลหรือนำไปใช้งานต่อ วิธีนี้แสดงให้เห็นว่า Node-RED สามารถทำหน้าที่เป็นมาสเตอร์ Modbus TCP อ่านค่าจากเกตเวย์ได้โดยตรง ควบคู่ไปกับเส้นทางข้อมูลผ่าน MQTT ดังแสดงในภาพที่ 3.13



ภาพที่ 3.13 โฟลว์ Node-RED สำหรับอ่านค่ารีจิสเตอร์ HR0-HR3 จาก ESP32-S3 ผ่าน Modbus TCP (FC03) ทุก 5 วินาที

บทที่ 4

การประเมินผลและผลการทดลอง

บทนี้รายงานผลการประเมินสองส่วน ได้แก่ การทดสอบไลบรารีเกตเวย์ Modbus แบบโอเพนซอร์สสำหรับ ESP32 ในเชิงประจักษ์ และการวัดสมรรถนะด้านทรัพยากรและเวลาของเกตเวย์ฉบับใช้งานจริงที่พัฒนาขึ้นจากไลบรารีที่คัดเลือกไว้

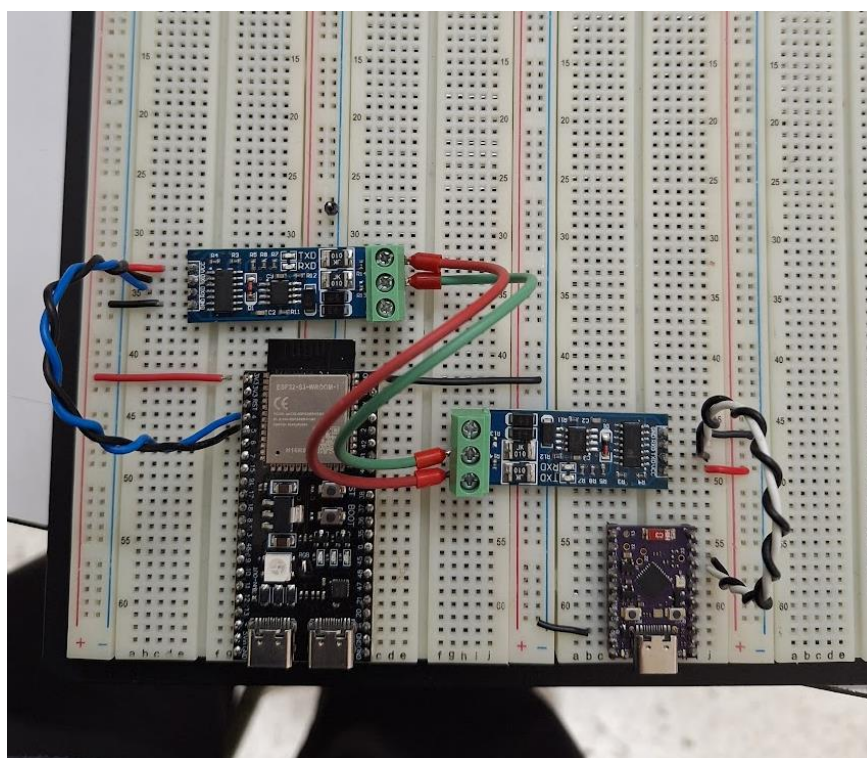
4.1 การประเมินไลบรารีเกตเวย์ Modbus

นอกเหนือจากบทบาทเกตเวย์ Modbus ที่เป็นเป้าหมายหลักของการประเมินในหัวข้อนี้แล้ว เฟิร์มแวร์ยังทำหน้าที่ส่งข้อมูลโทรมาตรขึ้นไปยังชุด TIG ผ่านโพรโทคอล MQTT ควบคู่กันไปด้วย ในแต่ละรอบการสำรวจ (ทุก 2 วินาที) เฟิร์มแวร์จะรวบรวมค่าที่อ่านได้จากอุปกรณ์ทั้งหมดเป็นเอกสาร JSON หนึ่งชุด ประกอบด้วยอุณหภูมิและความชื้นจาก XY-MD02 ค่าทางไฟฟ้าจากมิเตอร์ ได้แก่ แรงดันไฟฟ้า กระแสไฟฟ้า กำลังไฟฟ้าที่ใช้งานจริง กำลังไฟฟ้าที่ปรากฏ กำลังไฟฟ้ารีแอกทีฟ ค่าตัวประกอบกำลังไฟฟ้า ความถี่ และพลังงานสะสม (kWh) พร้อมทั้งรหัสสถานะและตัวนับการสำรวจของแต่ละอุปกรณ์ จากนั้นจึงเผยแพร่ไปยังหัวข้อ `sensors/esp32/data` บนโบรกเกอร์ Mosquitto ด้วย QoS 1 หากอุปกรณ์ใดอ่านค่าไม่สำเร็จ ฟิลด์ค่าของอุปกรณ์นั้นจะถูกละไว้และส่งเฉพาะรหัสสถานะ เพื่อไม่ให้ข้อมูลที่ผิดพลาดปะปนเข้าไปในฐานข้อมูล ทางฝั่ง Raspberry Pi ปลั๊กอินอินพุต `mqtt_consumer` ของ Telegraf ที่สมัครรับข้อความในหัวข้อดังกล่าวไว้ จะแปลงฟิลด์ JSON เหล่านี้เป็นจุดข้อมูล แล้วเขียนลงบั๊กเก็ต `sensors` ของ InfluxDB จากนั้น Grafana จะสืบค้นไปแสดงผลบนแดชบอร์ดแบบเรียลไทม์ ดังแสดงในหัวข้อ 4.2 เส้นทางข้อมูลโทรมาตรนี้ทำงานอยู่เบื้องหลังตลอดการทดสอบ และเป็นภาระงานจริงที่เฟิร์มแวร์ต้องรองรับควบคู่ไปกับบทบาทเกตเวย์ที่ถูกประเมินในหัวข้อนี้

4.1.1 การตั้งค่าชุดทดลองและวิธีการ

งานนี้ประเมินไลบรารีเกตเวย์ Modbus แบบโอเพนซอร์สสำหรับ ESP32 จำนวนแปดตัว ได้แก่ `vwetter`, `zivillian`, `harihanv`, `tobiasfaust/SolaxModbusGateway`, `eModbus/eModbus`, `maxx-ukoo/esp32-modbus-tcp2rtu`, `espressif/esp-modbus` และ `emelianov/modbus-esp8266` แต่ละไลบรารีถูกบิลด์ในสภาพแวดล้อม PlatformIO แยกกัน แต่ใช้บอร์ด ESP32-S3-DevKitC-1 (N16R8) ตัวเดียวกันร่วมกับตัวรับส่งสัญญาณ RS-485 รุ่น MAX3485 ฝั่งเครื่องคอมพิวเตอร์โฮสต์รัน

pymodbus 3.6 ทำหน้าที่เป็นทั้งไคลเอนต์ Modbus TCP และสเลฟ RTU ไปพร้อมกันที่ 9,600 baud, 8-N-1 การวัดเวลาแฝงแบบไปกลับใช้ลำดับการอ่าน-เขียนสี่ขั้นตอนกับอุปกรณ์หนึ่งตัว ได้แก่ อ่านรีจิสเตอร์ 0-4 ด้วย FC03 เขียนรีจิสเตอร์เดี่ยวด้วย FC06 อ่านค่ากลับมาตรวจสอบด้วย FC03 และเขียนคืนค่าทั้งบล็อกด้วย FC16 แล้วรายงานผลเป็นค่าเฉลี่ยของทุกขั้นตอน ส่วนปริมาณการใช้แฟลชและแรมอ่านจากผลการบิลด์ของแต่ละไลบรารีโดยตรง นอกจากนี้ทุกไลบรารียังต้องผ่านชุดทดสอบ stress test ที่จำลองสถานการณ์ผิดปกติ ทั้งการฉีดค่า CRC ที่ผิดพลาด การหน่วงเวลาตอบกลับของสเลฟ 500 มิลลิวินาที การรีเซ็ตการเชื่อมต่อ TCP เมื่อสเลฟหยุดทำงาน และการหลุดของ Wi-Fi พร้อมการเชื่อมต่อกลับอัตโนมัติ ทั้งนี้เวลาส่งข้อมูลบนสายสัญญาณตามทฤษฎี (wire-time floor) ของการรับส่งไปกลับหนึ่งรอบสำหรับเฟรมทดสอบนี้อยู่ที่ประมาณ 24 มิลลิวินาที (ค่าขอ 8 ไบต์ รวมกับการตอบกลับ 15 ไบต์ ที่ 9,600 baud, 8-N-1) ชุดทดลองที่ใช้แสดงในภาพที่ 4.1



ภาพที่ 4.1 ชุดทดลองการศึกษา A บนเบรคบอร์ด: ESP32-S3 ทำหน้าที่เกตเวย์ Modbus RTU/TCP ร่วมกับตัวรับส่งสัญญาณ MAX3485 และ ESP32-C6 จำลองสเลฟ

นอกเหนือจากการจับเวลาโดยตรงแล้ว ยังมีการใช้เกณฑ์เชิงคุณภาพอีกสืบข้อในการประเมิน ได้แก่ ความเคลื่อนไหวของ repository ความนิยมในการใช้งาน การรองรับเฟรมเวิร์กอินเทอร์เฟซด้านเครือข่าย คุณสมบัติของโพรโทคอล Modbus สถาปัตยกรรมของซอฟต์แวร์ การพึ่งพาไลบรารีภายนอก ระบบบิลด์ เอกสารประกอบ และคุณสมบัติสำหรับการนำไปใช้งานจริง

4.1.2 ผลการทดลอง

ตารางที่ 4.1 เวลาแฝงไปกลับของ FC03 และการใช้ทรัพยากรต่อไลบรารี Modbus (ESP32-S3, 100 รีจิสเตอร์, 9,600 baud) เวลาแฝงหน่วยเป็นมิลลิวินาที แฟลชและแรมหน่วยเป็นกิโลไบต์

ไลบรารี	เฟรมเวิร์ก	เฉลี่ย	p95	สูงสุด	แฟลช	แรม	stress test
tobiasfaust (raw RTU)	Arduino	27.7	34.2	34.2	300	22.0	ผ่าน
vwetter (raw RTU)	Arduino	27.9	34.2	34.2	301	22.0	ผ่าน
harihanv (raw RTU)	Arduino	28.0	34.2	34.2	300	22.0	ผ่าน
maxx-ukoo/esp-modbus	ESP-IDF	27.8	34.4	34.5	270	14.4	ผ่าน
espressif/esp-modbus	ESP-IDF	28.0	34.1	34.3	270	14.4	ผ่าน
eModbus/eModbus	Arduino	30.3	36.0	36.0	384	22.9	ผ่าน
zivillian/eModbus	Arduino	32.9	36.0	132.3	384	22.9	ผ่าน
emelianov/modbus-esp8266	Arduino	33.4	40.0	40.0	310	22.2	ผ่าน (แนะนำ)

ไลบรารีทั้งแปดตัวผ่านชุดทดสอบ stress test ได้ทั้งหมด ไลบรารีที่มีเวลาแฝงเฉลี่ยต่ำที่สุดคือ tobiasfaust (27.7 มิลลิวินาที สูงกว่าขีดจำกัดเวลาบนสายสัญญาณราว 4 มิลลิวินาที) ส่วนไลบรารีในตระกูล ESP-IDF (esp-modbus และ maxx-ukoo) เข้าใกล้ความเร็วของสายสัญญาณและใช้แรมน้อยที่สุด (14.4 KB) เนื่องจากสแตก Modbus ของ IDF ทำงานอยู่ในทาสก์ของตัวเอง สำหรับไลบรารีฝั่ง Arduino แบบอะซิงโครนัส (eModbus และ zivillian) มีเวลาแฝงเพิ่มขึ้นราว 2 มิลลิวินาทีต่อขั้นตอนการรอบการสำรวจของ FreeRTOS ที่ตั้งไว้ 2 มิลลิวินาที และ zivillian ยังปรากฏค่าผิดปกติ (outlier) หนึ่งค่าที่ 132 มิลลิวินาที ขณะที่ emelianov/modbus-esp8266 มีเวลาแฝงเฉลี่ยสูงที่สุด (33.4 มิลลิวินาที)

4.1.3 ข้อเสนอแนะ

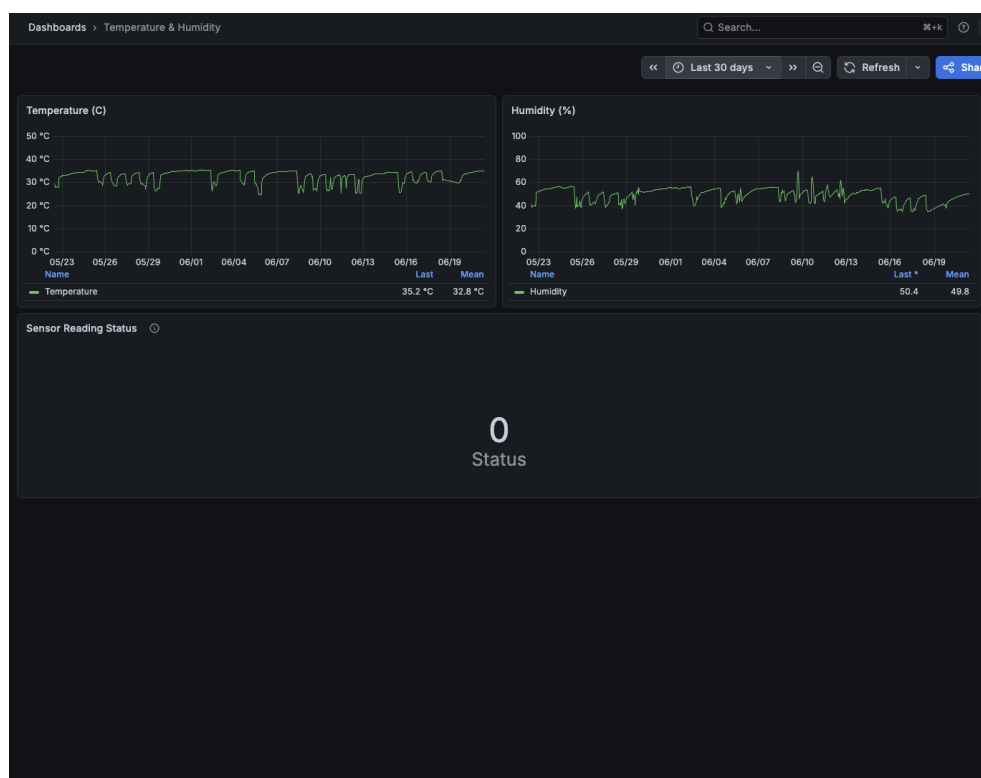
เวลาแฝงไม่ใช่เกณฑ์ตัดสินหลัก เพราะทุกตัวเลือกล้วนอยู่ห่างจากขีดจำกัดเวลาบนสายสัญญาณเพียงประมาณ 10 มิลลิวินาที และต่ำกว่ารอบส่งข้อมูลโทรมาตรสองวินาทีของแพลตฟอร์มอย่างมาก การคัดเลือกจึงให้น้ำหนักไปที่ความสามารถในการบำรุงรักษา ความครอบคลุมของคุณสมบัติ และความทนทานเป็นหลัก ไอบรรีที่ได้รับการแนะนำให้ใช้เป็นเกตเวย์ที่ใช้งานจริงคือ emelianov/modbus-esp8266 เนื่องจากเป็นไอบรรีเพียงตัวเดียวที่ยังมีการดูแลรักษาอย่างต่อเนื่อง ณ ช่วงเวลาที่ประเมิน อีกทั้งเป็นหนึ่งในสองไอบรรี (ร่วมกับ eModbus) ที่รองรับทั้งสามบทบาทที่แพลตฟอร์มนี้ต้องการพร้อมกัน คือ มาสเตอร์ RTU, สเลฟ Modbus TCP และการรองรับไคลเอนต์ TCP หลายตัว นอกจากนี้ยังให้ไอบรรีขนาดเล็กที่สุดในบรรดาไอบรรีที่รองรับ TCP (310 KB) ในขณะที่ผ่านชุดทดสอบ stress test ได้ครบถ้วน ส่วน eModbus/eModbus เป็นทางเลือกที่เหมาะสมสำหรับงานที่ออกแบบด้วยทาสก์ FreeRTOS หรือรองรับไคลเอนต์ TCP พร้อมกันมากกว่าห้าตัว

4.2 การวัดสมรรถนะของเกตเวย์

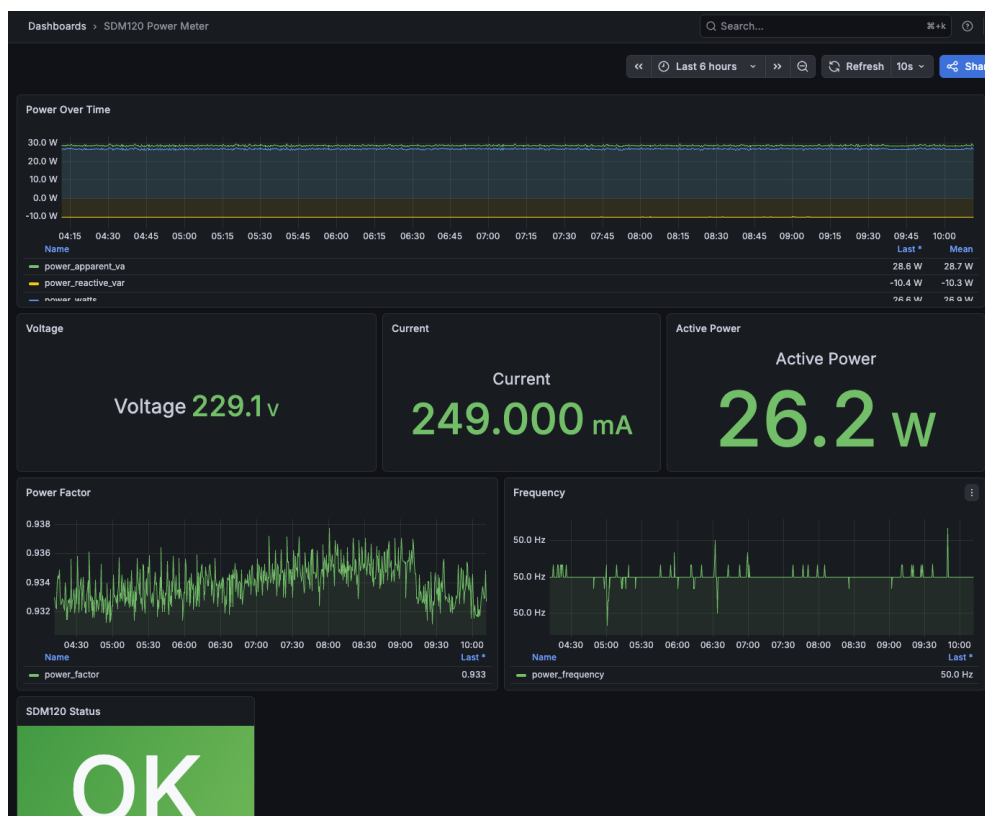
การทดสอบข้างต้นวัดไอบรรีแต่ละตัวแบบแยกกัน แต่เฟิร์มแวร์เกตเวย์ที่ใช้งานจริงต้องทำงานมากกว่านั้น เพราะรันไอบรรี emelianov ที่คัดเลือกไว้ในบทบาทสเลฟ Modbus TCP ควบคู่กับมาสเตอร์ RTU ที่เขียนขึ้นเอง ผู้เผยแพร่ MQTT และวอตซ์ด็อก โดยทั้งหมดอยู่ในรูปแบบรวมมือกัน เมื่อพัฒนาในลักษณะนี้ เฟิร์มแวร์ใช้พื้นที่แฟลช 946 KB จากพาร์ทิชันแอปพลิเคชัน 1,310 KB (ร้อยละ 72.2) และใช้แรม 48.5 KB จาก 327.6 KB (ร้อยละ 14.8) ผลนี้ยืนยันว่าเกตเวย์ที่รองรับสองโพรโทคอลอย่างสมบูรณ์เหมาะสมกับ MCU ต้นทุนต่ำ โดยยังมีทรัพยากรเหลืออีกมาก งานทุกส่วน ทั้งการสำรวจเซนเซอร์ RTU การอัปเดตรีจิสเตอร์ การเผยแพร่ผ่าน MQTT และการให้บริการสเลฟ TCP ทำเสร็จภายในรอบสองวินาที โดยมีเวลาเหลือมาก การออกแบบแบบรวมมือกันนี้ ทำให้ไม่มีบทบาทใดขัดขวางบทบาทอื่น เกตเวย์จึงบรรจุเป้าหมายเชิงหน้าที่ คือ เชื่อมอุปกรณ์ไอทีอนุกรมเดิมเข้ากับแดชบอร์ดฝั่งไอที (ผ่าน MQTT) และโฮสต์ควบคุมแบบไอพี (ผ่าน Modbus TCP) ได้ภายในขีดจำกัดทรัพยากรของบอร์ดราคาประหยัดเพียงบอร์ดเดียว ดังแสดงในภาพที่ 4.2 และภาพที่ 4.3 นอกจากนี้ ภาพที่ 4.4 และภาพที่ 4.5 แสดงระบบขณะทำงานจริง ได้แก่ แดชบอร์ดเฝ้าติดตามทรัพยากรของ Raspberry Pi และจอสัมผัสที่แสดงผล Grafana ในโหมดถือออสก์

จากการตรวจสอบเพิ่มเติมในที่เก็บโค้ดของไอบรรี emelianov/modbus-esp8266 [9] ยืนยันว่าไอบรรีที่คัดเลือกสามารถทำหน้าที่เกตเวย์ Modbus TCP/RTU ได้จริง ไอบรรีรองรับการสร้างอินสแตนซ์ ModbusRTU และ ModbusTCP หลายตัวให้ทำงานร่วมกันในบทบาทใดก็ได้พร้อมกัน

และตั้งแต่รุ่น 4.1.0 ได้เพิ่มความสามารถประมวลผลเฟรม Modbus ระดับดิบ (raw frame processing) ผ่านฟังก์ชัน `onRaw()`, `rawRequest()` และ `rawResponse()` พร้อมตัวอย่างอย่างเป็นทางการใน โพลเดอร์ Bridge ที่สาธิตการเป็นบริดจ์แบบโปร่งใส (transparent bridge) ซึ่งส่งต่อเฟรมคำขอจาก โคลเอนต์ Modbus TCP ไปยังบัส RS-485 โดยตรง อย่างไรก็ตาม บริดจ์แบบโปร่งใสรองรับคำขอค้างอยู่ได้ครั้งละหนึ่งรายการตามธรรมชาติของบัสอนุกรม ตัวอย่างของไลบรารีจึงตอบรหัสข้อยกเว้น Slave Device Busy เมื่อมีคำขอใหม่เข้ามาระหว่างรอการตอบกลับจากฝั่ง RTU เฟิร์มแวร์ของแพลตฟอร์มนี้จึงเลือกใช้แนวทางสะท้อนค่ารีจิสเตอร์ (register mirror) แทน กล่าวคือ มาสเตอร์ RTU ดำรวจเซนเซอร์ตามรอบเวลาแล้วเก็บค่าล่าสุดไว้ใน holding registers HR0-HR20 ของฝั่งสเลฟ TCP ทำให้โคลเอนต์ TCP หลายตัวอ่านค่าได้พร้อมกันโดยไม่ต้องรอบัส RS-485 และจังหวะเวลาของบัสอนุกรมไม่ขึ้นกับจำนวนโคลเอนต์ฝั่งเครือข่าย ผลการตรวจสอบนี้ยืนยันว่าไลบรารีที่คัดเลือกรองรับการเป็นเกตเวย์ได้ทั้งสองรูปแบบ และแนวทางสะท้อนค่าที่ใช้อยู่เหมาะสมกับแพลตฟอร์มเพื่อการศึกษาที่มีผู้ใช้หลายคนอ่านค่าพร้อมกัน



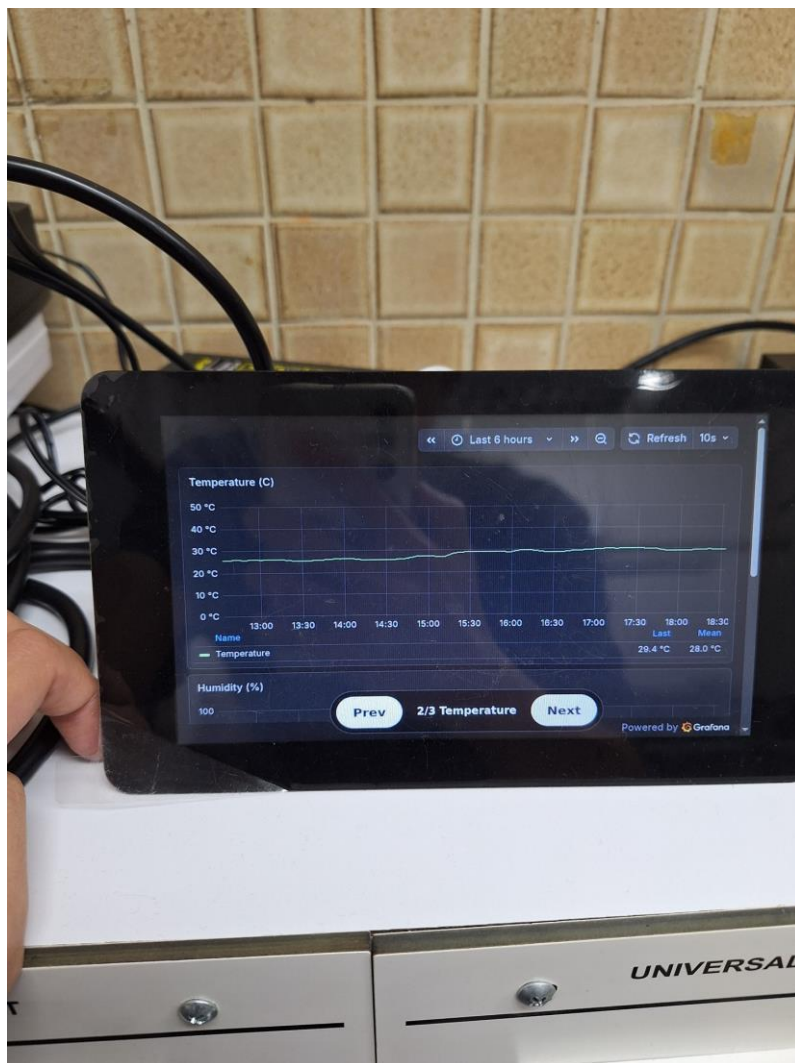
ภาพที่ 4.2 แดชบอร์ด Grafana แสดงอุณหภูมิและความชื้นจาก XY-MD02 ที่รับผ่านเกตเวย์แบบเรียลไทม์



ภาพที่ 4.3 แดชบอร์ด Grafana แสดงค่าทางไฟฟ้าจากมิเตอร์ SDM230 (แรงดันไฟฟ้า กระแสไฟฟ้า และกำลังไฟฟ้าจริง)



ภาพที่ 4.4 แดชบอร์ดเฝ้าติดตามทรัพยากรของ Raspberry Pi (อุณหภูมิซีพียู โหลด และหน่วยความจำ) ที่เก็บผ่าน Telegraf



ภาพที่ 4.5 จอสัมผัสขนาด 7 นิ้วของ Raspberry Pi แสดงแดชบอร์ด Grafana ในโหมดคีออสก์

บทที่ 5

การอภิปรายผลและบทสรุป

บทนี้สรุปภาพรวมของระบบที่พัฒนาขึ้น อภิปรายถึงปัญหาและอุปสรรคที่พบระหว่างการดำเนินงานพร้อมแนวทางแก้ไข และปิดท้ายด้วยข้อเสนอแนะสำหรับการพัฒนาต่อยอดในอนาคต

5.1 ภาพรวมระบบ

แพลตฟอร์มที่พัฒนาขึ้นในงานนี้ผสานการทำงานของตัวควบคุมทั้งสามประเภทภายในเครือข่ายท้องถิ่น (LAN) เดียวกัน ไมโครคอนโทรลเลอร์ ESP32-S3 ทำหน้าที่เป็นเอดจ์เกตเวย์แปลงโปรโตคอลจาก Modbus RTU ไปเป็น Modbus TCP และ MQTT คอมพิวเตอร์บอร์ดเดี่ยว Raspberry Pi 4 ทำหน้าที่เป็นฮับไอทีที่รันโบรกเกอร์ MQTT และชุด TIG อยู่ในคอนเทนเนอร์ ส่วนพีแอลซี Siemens LOGO! 24CE ทำหน้าที่ควบคุมกระบวนการที่ต้องการความแม่นยำเชิงเวลา ข้อมูลจากเครื่องมือวัดหน้างานจะถูกส่งผ่านเอดจ์เกตเวย์ไปยังระบบไอที เพื่อจัดเก็บและแสดงผลบนแดชบอร์ด Grafana แบบเรียลไทม์ ผลการทดลองยืนยันว่าไมโครคอนโทรลเลอร์ราคาต่ำเพียงตัวเดียวสามารถทำหน้าที่เป็นเอดจ์เกตเวย์ เพื่อเชื่อมต่อระบบไอทีแบบเดิมที่ใช้การสื่อสารแบบอนุกรมเข้ากับแดชบอร์ดไอทีสมัยใหม่ได้ แม้จะมีข้อจำกัดด้านทรัพยากรเชิงฮาร์ดแวร์ในการทำงานของเฟิร์มแวร์ ทั้งนี้ตารางที่ 5.1 สรุปเปรียบเทียบคุณลักษณะของตัวควบคุมทั้งสามประเภท ซึ่งแสดงให้เห็นว่าอุปกรณ์แต่ละประเภทมีจุดแข็งที่เหมาะสมกับบทบาทต่างกัน และให้ผลลัพธ์ที่ดีที่สุดเมื่อนำมาทำงานร่วมกัน

ตารางที่ 5.1 การเปรียบเทียบตัวควบคุมทั้งสามประเภทที่ใช้ในแพลตฟอร์ม

คุณลักษณะ	MCU — ESP32-S3	SBC — Raspberry Pi 4	PLC — LOGO! 24CE
ระบบปฏิบัติการ	ไม่มี	Linux เต็มรูปแบบ	เฟิร์มแวร์เฉพาะผู้ผลิต
ความแม่นยำเชิงเวลา	เรียลไทม์แบบไม่เข้มงวด	ไม่มีความแม่นยำเชิงเวลา	เรียลไทม์แบบเข้มงวด
I/O ในตัว	GPIO/ADC/UART/RS-485	GPIO + USB; ไม่มีแอนะล็อก	DI 24 V, AI 0-10 V
เครือข่าย	Wi-Fi / BLE	กิกะบิตอีเทอร์เน็ต	Modbus TCP 10/100
แรงดันไฟฟ้าจ่าย	7-36 V DC (ลอจิก 3.3 V)	5 V DC	24 V DC

คุณลักษณะ	MCU — ESP32-S3	SBC — Raspberry Pi 4	PLC — LOGO! 24CE
การใช้พลังงาน	~0.6-1.7 W	~3.4-7.6 W	<=4.5 W ที่ 24 V DC
ชุดเครื่องมือ	เปิด (Arduino/PlatformIO)	เปิด (Linux/Docker)	เฉพาะผู้ผลิต (Soft Comfort)
การรับรองมาตรฐาน	ไม่มี	ไม่มี	CE, UL, FM, ATEX
ต้นทุนโดยประมาณ	~1,895 บาท	~1,900-3,200 บาท	~5,000-6,500 บาท

MCU ให้ต้นทุนต่อช่องสัญญาณและการใช้พลังงานต่ำที่สุด และเหมาะกับการเป็นเกตเวย์ที่หน้างาน แต่ก็ขาดระบบปฏิบัติการและการรับรองมาตรฐาน ส่วน SBC มอบความหลากหลายของซอฟต์แวร์ที่หาอุปกรณ์อื่นเทียบได้ยาก เพราะสามารถรันแฟ้มเกจหรือคอนเทนเนอร์ Linux ใด ๆ ได้โดยไม่ต้องดัดแปลง อย่างไรก็ตามตัวจัดการงานแบบแทรกก่อน (preemptive) ของ SBC ไม่สามารถรับประกันพฤติกรรมเรียลไทม์แบบเข้มงวดได้ จึงเหมาะกับงานในชั้นข้อมูลมากกว่างานควบคุม ขณะที่ PLC เป็นอุปกรณ์เดียวที่ขับเคลื่อนแอกชูเอเตอร์ระดับแรงดันไฟหลักได้ ภายใต้รอบสแกนที่มีขอบเขตแน่นอนและผ่านการรับรอง ซึ่งถือเป็นคุณสมบัติด้านความปลอดภัยมากกว่าจะเป็นเพียงตัวเลขสมรรถนะ กระนั้นก็ยังเป็นอุปกรณ์ที่มีราคาสูงที่สุดและปิดที่สุด การนำอุปกรณ์ประเภทหนึ่งไปแทนที่อีกประเภทหนึ่งข้ามชั้น จึงเท่ากับแลกคุณสมบัติเหล่านี้ทิ้งไปโดยตรง การกำหนดบทบาทให้อุปกรณ์แต่ละประเภทจึงเป็นผลลัพธ์เชิงวิศวกรรม ไม่ใช่เรื่องของความชอบส่วนบุคคล

5.2 ปัญหา อุปสรรค และวิธีการแก้ไข

ระหว่างการพัฒนาและพัฒนางานนี้พบปัญหาและอุปสรรคหลายประการ พร้อมแนวทางแก้ไขดังนี้ ประการแรก ไลบรารีสำหรับสร้างเกตเวย์ Modbus แบบโอเพนซอร์สบน ESP32 มิให้เลือกจำนวนมาก และมีคุณภาพแตกต่างกัน ทั้งในแง่การดูแลรักษาและความครอบคลุมของคุณสมบัติ จึงตัดสินใจเลือกได้ยาก การแก้ไขคือกำหนดเกณฑ์การประเมินให้ชัดเจนและทดสอบสมรรถนะบนฮาร์ดแวร์จริงเพื่อคัดเลือกอย่างเป็นระบบ ดังรายงานไว้ในบทที่ 4 ประการที่สอง การสื่อสารผ่านบัส RS-485 มีความไวต่อการเดินสาย การสลับขั้ว A/B และการต่อความต้านทานปลายสาย ทำให้เกิดข้อผิดพลาด CRC เป็นระยะ การแก้ไขคือ ตรวจสอบการเดินสาย ต่อกราวด์ร่วม และตั้งค่า baud rate ให้ตรงกันทุกอุปกรณ์ ประการที่สาม Raspberry Pi 4 ที่ใช้ในงานนี้เป็นรุ่นหน่วยความจำ 2 GB แต่ต้องรองรับคอนเทนเนอร์หลายตัวพร้อมกัน ได้แก่ Mosquitto, InfluxDB, Telegraf, Grafana และ Node-RED หน่วยความจำจึงเป็นข้อจำกัดสำคัญของชั้นไอที การแก้ไขคือใช้ระบบปฏิบัติการ Raspberry Pi OS Lite ที่ไม่มีส่วนติดต่อกราฟิก จำกัดบริการให้เหลือเท่าที่จำเป็น และเฝ้าติดตามการใช้หน่วยความจำผ่านแดชบอร์ดสถานะของโฮสต์ (ภาพที่ 4.4) เพื่อยืนยัน

ว่าระบบมีหน่วยความจำเหลือเพียงพอขณะทำงานต่อเนื่อง ประการที่สี่ ในช่วงแรก Node-RED ถูกติดตั้งเป็นคอนเทนเนอร์แยกต่างหากนอกชุด Docker Compose ทำให้ไฟล์ใน Node-RED ไม่สามารถเชื่อมต่อ Modbus TCP ไปยังเกตเวย์ได้ เนื่องจากการตั้งค่าเครือข่ายของคอนเทนเนอร์ที่ติดตั้งแยกกันไม่สอดคล้องกับบริการส่วนที่เหลือ การแก้ไขคือย้าย Node-RED เข้ามาประกาศไว้ในไฟล์ Docker Compose เดียวกันกับบริการอื่นทั้งหมด เพื่อให้ทุกบริการอยู่บนเครือข่ายเดียวกันและจัดการร่วมกันได้ด้วยคำสั่งเดียว หลังจากนั้นการเชื่อมต่อ Modbus TCP จึงทำงานได้ตามปกติ

5.3 ข้อเสนอแนะ

งานนี้ยังสามารถพัฒนาต่อยอดได้อีกหลายแนวทาง ในด้านเอดจ์เกตเวย์ ควรเพิ่มบัฟเฟอร์แบบวงแหวน (ring buffer) สำหรับเก็บข้อมูลโทรมาตรไว้ชั่วคราวเพื่อรองรับช่วงที่ Wi-Fi ขาดหาย และเพิ่มกลไกการอัปเดตเฟิร์มแวร์ผ่านอากาศ (Over-the-Air: OTA) เพื่อให้บำรุงรักษาได้สะดวกขึ้น ในด้านชั้นไอที ควรเพิ่มการสำรองข้อมูล InfluxDB ตามกำหนดเวลา การตรวจสอบสุขภาพของคอนเทนเนอร์ และการแจ้งเตือนเมื่อค่าถึงขีดจำกัดใน Grafana เพื่อเสริมความน่าเชื่อถือของชั้นข้อมูล ในด้านการสื่อสาร ควรศึกษาการเพิ่มโปรโตคอล OPC-UA เพื่อขยายความสามารถในการทำงานร่วมกับระบบอุตสาหกรรมอื่น ๆ และในด้านการนำไปใช้งาน ควรนำแพลตฟอร์มนี้ไปทดลองใช้จริงในรายวิชาปฏิบัติการร่วมกับนักศึกษา เพื่อประเมินคุณค่าทางการศึกษา ตลอดจนต่อยอดไปสู่การใช้งานจริงหรืองานวิจัยในระดับที่สูงขึ้นต่อไป

เอกสารอ้างอิง

- [1] S. Santos, P. Costa, and A. Rocha, “IT/OT convergence in Industry 4.0: risks and analysis of the problems,” in Proc. 18th Iberian Conf. on Information Systems and Technologies (CISTI), 2023.
- [2] L. D. Xu, E. L. Xu, and L. Li, “Industry 4.0: state of the art and future trends,” Int. J. Prod. Res., vol. 56, no. 8, pp. 2941-2962, 2018.
- [3] Enterprise-Control System Integration - Part 1: Models and Terminology, ISA-95.00.01, 2010.
- [4] Modbus Organization, Modbus Application Protocol Specification V1.1b3, 2012.
- [5] Information technology - Message Queuing Telemetry Transport (MQTT) v3.1.1, ISO/IEC 20922, 2016.
- [6] InfluxData, “Telegraf 1.28 documentation,” 2024. [Online]. Available: <https://docs.influxdata.com/telegraf/>. (Accessed: Jun. 2026).
- [7] InfluxData, “InfluxDB 2.7 documentation,” 2024. [Online]. Available: <https://docs.influxdata.com/influxdb/v2/>. (Accessed: Jun. 2026).
- [8] Grafana Labs, “Grafana documentation,” 2024. [Online]. Available: <https://grafana.com/docs/grafana/latest/>. (Accessed: Jun. 2026).
- [9] A. Emelianov, “modbus-esp8266: Modbus Library for Arduino,” GitHub repository, 2023. [Online]. Available: <https://github.com/emelianov/modbus-esp8266>. (Accessed: Jul. 2026).

ประวัติผู้แต่ง



พัทธนันท์ พันธุมณี เกิดเมื่อวันที่ 28 กันยายน ค.ศ. 2001 ที่จังหวัดสระบุรี สำเร็จการศึกษาระดับประกาศนียบัตรวิชาชีพ โรงเรียนเตรียมวิศวกรรมศาสตร์ ไทย-เยอรมัน คณะวิทยาลัยเทคโนโลยีอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ จังหวัดกรุงเทพมหานคร ปีการศึกษา 2562 และ สำเร็จการศึกษาระดับปริญญาตรี สาขาวิชาวิศวกรรมไฟฟ้า ภาควิชา วิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระ นครเหนือ จังหวัดกรุงเทพมหานคร ปีการศึกษา 2568

ภาคผนวก ก

ภาพรวมการออกแบบเฟิร์มแวร์ ESP32-S3

ภาคผนวกนี้สรุปการออกแบบเฟิร์มแวร์ของเอจเกตเวย์ Waveshare ESP32-S3 Relay 6CH ในระดับโมดูลและหน้าที่ความรับผิดชอบ โดยตั้งใจไม่นำซอร์สโค้ดฉบับเต็มมาแสดง ทั้งนี้ซอร์สโค้ดฉบับเต็มของเฟิร์มแวร์เอจเกตเวย์ ESP32-S3 ที่กล่าวถึงในภาคผนวกนี้เผยแพร่ไว้ในที่เก็บโค้ด GitHub ของโครงการที่ https://github.com/patthananb/Lab_network_monitoring เฟิร์มแวร์อยู่ในไลบรารี `firmware_platformio/` ซึ่งเป็นโครงการ PlatformIO บน Arduino core ประกอบด้วยไฟล์ `platformio.ini` และโมดูลใน `src/` ได้แก่ `main.cpp`, `modbus_rtu`, `sensors`, `outputs`, `telemetry_types` และ `firmware_config.h` พร้อมไฟล์ตัวอย่าง `secrets.h.example` สำหรับกำหนดค่า Wi-Fi และ โพรกเกอร์ นอกจากนี้ยังมีสำเนาในรูปแบบสเก็ตซ์ Arduino IDE อยู่ในไลบรารี `firmware/` เฟิร์มแวร์นี้เป็นแอปพลิเคชันบน Arduino core ที่พัฒนาด้วย PlatformIO โดยจัดวางการทำงานเป็นโมดูลแบบรวมมือกันที่ไม่บล็อกการทำงาน วนทำงานทุก ๆ สองวินาที และไม่เรียกใช้คำสั่ง `delay()` เลย ทั้งนี้เพื่อไม่ให้บทบาทใดไปขัดขวางบทบาทอื่น

ก.1 บทบาทที่ทำงานพร้อมกัน

ภายในลูปนี้ให้บริการสับบทบาทเรียงตามลำดับ โดยแต่ละบทบาทถูกเขียนขึ้นเป็นอ็อบเจกต์สถานะขนาดเล็กที่ถูกเรียกสำรวจหนึ่งครั้งต่อรอบการทำงาน ดังนี้

มาสเตอร์ Modbus RTU - สำรวจ XY-MD02 (FC04) และมิเตอร์ไฟฟ้า SDM230 (FC03//FC04) ผ่าน RS-485 ประกอบเฟรมคำขอแต่ละคำสั่งด้วย CRC-16 ตรวจจบการตอบสนองแบบซิงโครไนซ์ (รหัสฟังก์ชันที่มีบิตสูงสุดถูกตั้งค่า) และบันทึกรหัสสถานะรวมต่อสเลฟแทนการโยนข้อผิดพลาดออกมา

ตัวเผยแพร่ MQTT - แปลงค่าที่อ่านได้ล่าสุดเป็นเอกสาร JSON ขนาดกะทัดรัดและเผยแพร่ที่ QoS 1 เมื่อเกิดข้อผิดพลาดฝั่งอุปกรณ์ ฟิลด์ค่าที่ได้รับผลกระทบจะถูกละไว้และส่งเฉพาะรหัสสถานะเท่านั้น เพื่อไม่ให้ปลายทางที่รับข้อมูลรับค่าที่ไม่ถูกต้อง

สเลฟ Modbus TCP - ให้บริการค่าที่อ่านได้ชุดเดียวกันบนพอร์ต TCP 502 เป็น holding registers HR0-HR20 เพื่อให้โฮสต์ไอพีใดก็ตาม (รวมถึง PLC ที่ทำหน้าที่เป็นมาสเตอร์) สามารถอ่านค่าได้โดยไม่ต้องเตะบัสอนุกรม

วอตช์ด็อก - ตัวจับเวลาซอฟต์แวร์ที่ตรวจจ็อบรอบที่ค้างและบังคับให้รีเซ็ตใหม่ ป้องกันตัวรับส่งสัญญาณหรือซ็อกเก็ตโพรกเกอร์ที่ค้าง

ก.2 แผนผัง Holding Register (HR0-HR20)

บทบาตสเลฟ TCP ให้บริการแผนผังรีจิสเตอร์แบบตายตัว โดยใช้ค่าจำนวนเต็มที่ปรับสเกลแล้วเพื่อหลีกเลี่ยงการส่งจุดทศนิยมบนสายสัญญาณสำหรับเซนเซอร์อย่างง่าย ส่วนค่าทศนิยม IEEE-754 ของ SDM230 จะถูกเผยแพร่ในรูปของคู่ word สูง/ต่ำ

ตารางที่ ก.1 แผนผัง Holding Register (HR0-HR20) ของบทบาตสเลฟ TCP

รีจิสเตอร์	เนื้อหา	การเข้ารหัส
HR0	อุณหภูมิ	จำนวนเต็มมีเครื่องหมาย, ฐาน 100 องศาเซลเซียส
HR1	ความชื้นสัมพัทธ์	จำนวนเต็มไม่มีเครื่องหมาย, ฐาน 100 %RH
HR2	รหัสสถานะต่อสเลฟ	enum (OK / CRC / timeout / exception)
HR3	ตัวนับการสำรวจ	จำนวนเต็มไม่มีเครื่องหมาย, วนกลับเมื่อสิ้น
HR4-HR19	ค่าที่วัดจาก SDM230	ทศนิยม IEEE-754 8 ค่า, คู่ word สูง/ต่ำ
HR20	เวลาทำงานของเกตเวย์	วินาที

ก.3 การใช้ทรัพยากรจากการบิลด์

เมื่อพัฒนาด้วย Arduino core บน FreeRTOS ผ่าน PlatformIO เฟิร์มแวร์ที่ใช้งานจริงใช้พื้นที่แฟลช 946 KB จากพาร์ทิชันแอปพลิเคชัน 1,310 KB (ร้อยละ 72.2) และใช้แรม 48.5 KB จาก 327.6 KB (ร้อยละ 14.8) ตัวเลขนี้ยืนยันว่าเกตเวย์ที่รองรับสองโพรโทคอลอย่างสมบูรณ์นี้เหมาะสมกับ MCU ต้นทุนต่ำ ทั้งนี้ยังมีทรัพยากรเหลืออีกมากสำหรับคุณสมบัติในอนาคต เช่น การอัปเดตผ่านอากาศ (OTA Update) หรือบัฟเฟอร์แบบวงแหวน (Ring Buffer) สำหรับข้อมูลโทรมาตร

ภาคผนวก ข

การกำหนดค่า Docker Compose และ Telegraf

ภาคผนวกนี้อธิบายรายละเอียดของซอฟต์แวร์ชั้นไอทีที่ติดตั้งบน Raspberry Pi 4 ซึ่งทุกบริการทำงานอยู่ในรูปแบบคอนเทนเนอร์ (container) โดยสรุปเฉพาะหน้าที่และการตั้งค่าที่สำคัญของแต่ละบริการ ส่วนไฟล์ docker-compose.yml, telegraf.conf และ mosquitto.conf ฉบับเต็มเผยแพร่ไว้ในไดเรกทอรี docker/ และ weatherstation_docker/ ของที่เก็บโค้ด GitHub เดียวกัน (https://github.com/patthanab/Lab_network_monitoring) โดยชุดหลังมีไฟล์ .env.example สำหรับตัวแปรสภาพแวดล้อม และเทมเพลตแดชบอร์ด Grafana ในรูปแบบ JSON ทั้งข้อมูลสภาพอากาศ มิเตอร์ไฟฟ้า และสถานะของ Raspberry Pi นอกจากนี้ที่เก็บโค้ดยังมีแดชบอร์ดเพิ่มเติมใน grafana/dashboards/ สคริปต์จอแสดงผลแบบกiosk ใน kiosk/ และเอกสารประกอบใน docs/ ทุกบริการใช้อิมเมจ Docker แบบโอเพนซอร์ส และถูกประกาศไว้ในไฟล์ Docker Compose เพียงไฟล์เดียว โดยระบุแท็กเวอร์ชันของอิมเมจไว้อย่างชัดเจน และกำหนดค่าความลับ (secret) ผ่านตัวแปรสภาพแวดล้อม (environment variable) ด้วยวิธีนี้ผู้ที่นำไปใช้งานต่อจึงสามารถติดตั้งซอฟต์แวร์ทั้งหมดได้ด้วยคำสั่งเพียงคำสั่งเดียว

ข.1 หน้าที่ของบริการแต่ละตัว

- Mosquitto 2.0 - โบรกเกอร์ MQTT (ตัวกลางรับส่งข้อความ) ทำหน้าที่รับข้อความที่เอดจ์เกตเวย์เผยแพร่เข้ามาทางพอร์ต TCP 1883 และรองรับการเชื่อมต่อจากเว็บเบราว์เซอร์ผ่าน WebSocket ทางพอร์ต 9001 พร้อมเปิดใช้การหมุนเวียนไฟล์ log (log rotation) ของ Docker เพื่อจำกัดการใช้พื้นที่ดิสก์
- InfluxDB 2.7 - ฐานข้อมูลอนุกรมเวลา (time-series database) ทำหน้าที่จัดเก็บข้อมูลโทรมาตรไว้ในบั๊กเก็ต (bucket) ชื่อ sensors ภายใต้องค์กรชื่อ weatherstation และสืบค้นข้อมูลด้วยภาษา Flux
- Telegraf 1.28 - เอเจนต์เก็บรวบรวมข้อมูล (data collection agent) ทำหน้าที่เชื่อม Mosquitto เข้ากับ InfluxDB โดยปลั๊กอินอินพุต mqtt_consumer สมัครรับข้อความจากโบรกเกอร์ที่ QoS 1 แล้วแปลงข้อมูล JSON จากนั้นปลั๊กอินเอาต์พุต influxdb_v2 เขียนข้อมูลลงฐานข้อมูล นอกจากนี้ยังใช้ปลั๊กอินอินพุตเพิ่มเติมเก็บข้อมูลสถานะของ Raspberry Pi ได้แก่ ซีพียู หน่วยความจำ ดิสก์ อุณหภูมิ และสถานะการเชื่อมต่อเครือข่าย

- Grafana - แดชบอร์ดแสดงผลข้อมูล แบ่งเป็นหน้าสำหรับข้อมูลสภาพอากาศ ค่าทางไฟฟ้า สถานะของเครื่องโฮสต์ และเครือข่าย พร้อมเปิดสิทธิ์ผู้ชมแบบอ่านอย่างเดียวโดยไม่ต้องเข้าสู่ระบบ (anonymous viewer) สำหรับจอแสดงผลแบบคีออสก์ (kiosk) ที่ทำงานโดยไม่มีผู้ดูแล

ข.2 สรุปพอร์ตและบริการที่เปิดใช้งาน

ตารางต่อไปนี้เป็นสรุปบริการทั้งหมดที่เปิดให้เข้าถึงได้ผ่านเครือข่าย ตามที่ติดตั้งใช้งานจริงบน Raspberry Pi 4 (ส่วนการเข้าถึงจากภายนอกผ่าน tunnel นั้นได้พิจารณาไว้เช่นกัน แต่ไม่รวมอยู่ในขอบเขตของระบบภายในเครือข่ายท้องถิ่นที่อธิบายในปฏิญานี้)

ตารางที่ ข.1 สรุปพอร์ตและบริการที่เปิดใช้งานบน Raspberry Pi

พอร์ต	บริการ	ประเภท	อิมเมจ/โปรเซส
TCP 1883	Mosquitto MQTT	Docker	eclipse-mosquitto:2.0
TCP 9001	Mosquitto WebSocket	Docker	บริดจ์ WebSocket
TCP 8086	InfluxDB	Docker	influxdb:2.7
UDP/TCP 8092/8094	พอร์ตอินพุตของ Telegraf	Docker	telegraf:1.28
TCP 3000	Grafana	Docker	grafana/grafana
TCP 1880	Node-RED (การสร้างต้นแบบ)	Docker	nodered/node-red
TCP 22	SSH	ระบบ	การจัดการ โฮสต์

ภาคผนวก ค

ตารางคะแนนการประเมินไลบรารี Modbus

ภาคผนวกนี้บันทึกตารางคะแนนการประเมินฉบับเต็มที่ใช้เป็นพื้นฐานของการคัดเลือกในหัวข้อ 4.1 โดยประเมินไลบรารีเกตเวย์ Modbus แบบโอเพนซอร์สจำนวนแปดตัวบนฮาร์ดแวร์ชุดเดียวกัน นอกเหนือจากผลการวัดเวลาแฝง แฟลช แรม และการทดสอบ stress test ในตารางที่ 4.1 แล้ว แต่ละไลบรารียังได้รับการให้คะแนนตามเกณฑ์เชิงคุณภาพอีกสิบข้อ

ค.1 ตารางเกณฑ์เชิงคุณภาพ

เกณฑ์การให้คะแนน: Full = รองรับได้เต็มรูปแบบ, Part = รองรับบางส่วนหรือต้องแก้ปัญหาเฉพาะหน้า, None = ไม่รองรับ ทั้งนี้สถานะการบำรุงรักษาเป็นข้อมูล ณ วันที่ทำการประเมิน

ตารางที่ ค.1 คะแนนการประเมินเชิงคุณภาพของไลบรารีเกตเวย์ Modbus

ไลบรารี	เฟรมเวิร์ก	การบำรุงรักษา	RTU+TCP	เอกสาร	ขนาดการใช้ทรัพยากร
emelianov/modbus-esp8266	Arduino	เต็มรูปแบบ	เต็มรูปแบบ	เต็มรูปแบบ	เล็ก (แนะนำ)
eModbus/eModbus	Arduino	บางส่วน	เต็มรูปแบบ	เต็มรูปแบบ	กลาง
espressif/esp-modbus	ESP-IDF	เต็มรูปแบบ	เต็มรูปแบบ	เต็มรูปแบบ	เล็ก
maxx-ukoo/esp32-modbus-tcp2rtu	ESP-IDF	ไม่มี	เต็มรูปแบบ	บางส่วน	เล็ก
tobiasfaust/SolaxModbusGateway	Arduino	บางส่วน	เต็มรูปแบบ	บางส่วน	ใหญ่
vwetter/esp32-modbus-gateway	Arduino	บางส่วน	เต็มรูปแบบ	บางส่วน	กลาง
harihanv/esp32-modbus-gateway	Arduino	ไม่มี	เต็มรูปแบบ	บางส่วน	กลาง
zivillian/esp32-modbus-gateway	Arduino	บางส่วน	เต็มรูปแบบ	บางส่วน	ใหญ่

ค.2 เหตุผลในการคัดเลือก

เวลาแฝงไม่ใช่เกณฑ์ตัดสินหลัก เพราะทุกตัวเลือกล้วนอยู่ห่างจากขีดจำกัดเวลาบนสายสัญญาณแบบไปกลับที่ประมาณ 24 มิลลิวินาที เพียงราว 10 มิลลิวินาที ซึ่งต่ำกว่ารอบส่งข้อมูลโพรโทคอลสองวินาทีของแพลตฟอร์มอยู่มาก ดังนั้นการคัดเลือกจึงให้น้ำหนักกับความสามารถในการบำรุงรักษา การรองรับสามบทบาทที่แพลตฟอร์มต้องการ (มาสเตอร์ RTU, สเลฟ Modbus TCP และไคลเอนต์ TCP หลายตัว) ปริมาณการใช้ทรัพยากร และความทนทานต่อการทดสอบ stress test

แทน ผลคือ emelianov/modbus-esp8266 เป็นตัวเลือกที่ดีที่สุด ด้วยการผสมผสานระหว่างการดูแลรักษาอย่างต่อเนื่อง การรองรับครบทุกบทบาท ไบนารีที่เล็กที่สุดในบรรดาไลบรารีที่รองรับ TCP และการผ่านชุดทดสอบ stress test ได้อย่างครบถ้วน

ภาคผนวก ง

เครื่องมือ IoT-Sniffer และการใช้งานเพื่อตรวจสอบ

IoT-Sniffer เป็นเครื่องมือวิเคราะห์โพรโทคอลแบบเชิงรับที่ทำงานแบบเรียลไทม์ เขียนขึ้นด้วยภาษา Python และใช้ในโครงการนี้เป็นเครื่องมือตรวจสอบระดับสายสัญญาณของเกตเวย์และในการทดสอบไลบรารี จุดที่ต่างจากเครื่องมืออื่นที่ประสงค์ทั่วไปคือไม่จำเป็นต้องใช้แทป (tap) หรือพอร์ตมิเตอร์ แต่รัน AsyncSniffer ของ scrapy โดยตรงบนการ์ดเครือข่ายของเครื่องโฮสต์ผ่านตัวกรองล่วงหน้า BPF ในระดับเคอร์เนล จากนั้นจึงถอดรหัส Modbus TCP (พอร์ต 502) และ MQTT (พอร์ต 1883 และผ่าน WebSocket หลังจากถอดเฟรมตาม RFC 6455 ออก) ที่ขึ้นแอปพลิเคชัน ทั้งนี้ซอร์สโค้ดฉบับเต็มของเครื่องมือนี้เผยแพร่ไว้ในที่เก็บโค้ด GitHub <https://github.com/patthanab/IoT-sniffer>

ง.1 สถาปัตยกรรม

เซิร์ฟเวอร์สำหรับดักจับจะส่งแพ็กเก็ตแต่ละตัวไปยังตัวประกอบรวม TCP ตามคู่โพล์ (ต้นทางปลายทาง) ซึ่งจะจัดการกับเซกเมนต์ที่มาถึงไม่เรียงลำดับ จากนั้นไบต์ที่ประกอบรวมแล้วจะถูกป้อนเข้าตัวแยกวิเคราะห์ที่เขียนด้วย Python ล้วน ตัวแยกวิเคราะห์ Modbus จะถอดรหัสส่วนหัว MBAP ขนาดเจ็ดไบต์ (ทั้ง Transaction ID หมายเลขโพรโทคอล ความยาว และหมายเลขหน่วย) พร้อมรหัสฟังก์ชัน และตั้งค่าสถานะเป็นการตอบกลับแบบขอยกเว้นเมื่อบิตสูงสุดของรหัสฟังก์ชันถูกตั้งไว้ ส่วนตัวแยกวิเคราะห์ MQTT จะถอดรหัสประเภทของส่วนหัวแบบตายตัว นิบิลของแฟล็ก และฟิลด์ความยาวที่เหลือแบบความยาวผันแปร ครอบคลุมแพ็กเก็ต CONNECT, PUBLISH, PUBACK, SUBSCRIBE และแพ็กเก็ต keep-alive ทั้งนี้เฟรมที่ได้จะถูกบันทึกลง SQLite ในโหมด write-ahead-log และส่งไปยังแดชบอร์ดบนเบราว์เซอร์ผ่าน WebSocket พร้อมภาพรวมของตัวชี้วัดหนึ่งครั้งต่อวินาที

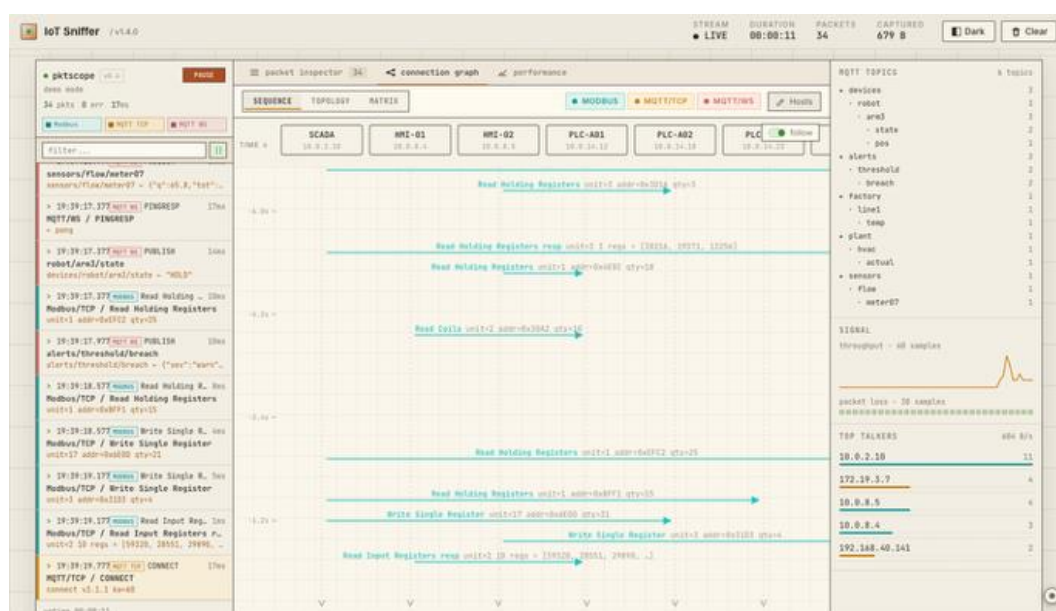
ง.2 บทบาทในโครงการนี้

เครื่องมือนี้ถูกใช้เพื่อยืนยันว่าไบต์ที่ปรากฏบนสายสัญญาณถูกต้อง ทั้งความถูกต้องของฟิลด์ MBAP และหมายเลขหน่วย การส่งค่าทศนิยม IEEE-754 ของ SDM230 ในลำดับ word ที่คาดหวัง และการที่คำขอ FC03 แต่ละคำสั่งจากไคลเอนต์จับคู่กับการตอบกลับจากเกตเวย์พอดีหนึ่งครั้ง การอ่านคู่คำขอ/การตอบกลับที่ถอดรหัสแล้วบนสายสัญญาณเช่นนี้ ถือเป็นการตรวจสอบที่เป็นอิสระจากการจับเวลาฝั่งโฮสต์ที่ใช้ในการทดสอบไลบรารีในหัวข้อ 4.1 นอกจากนี้ยังช่วยวินิจฉัย

ปัญหาเรื่องการประกอบเฟรมและลำดับไบนารีได้ตั้งแต่ระหว่างการพัฒนา ก่อนที่ปัญหาเหล่านั้นจะลุกลามไปทำให้ข้อมูลโทรมาตรที่จัดเก็บไว้เสียหาย

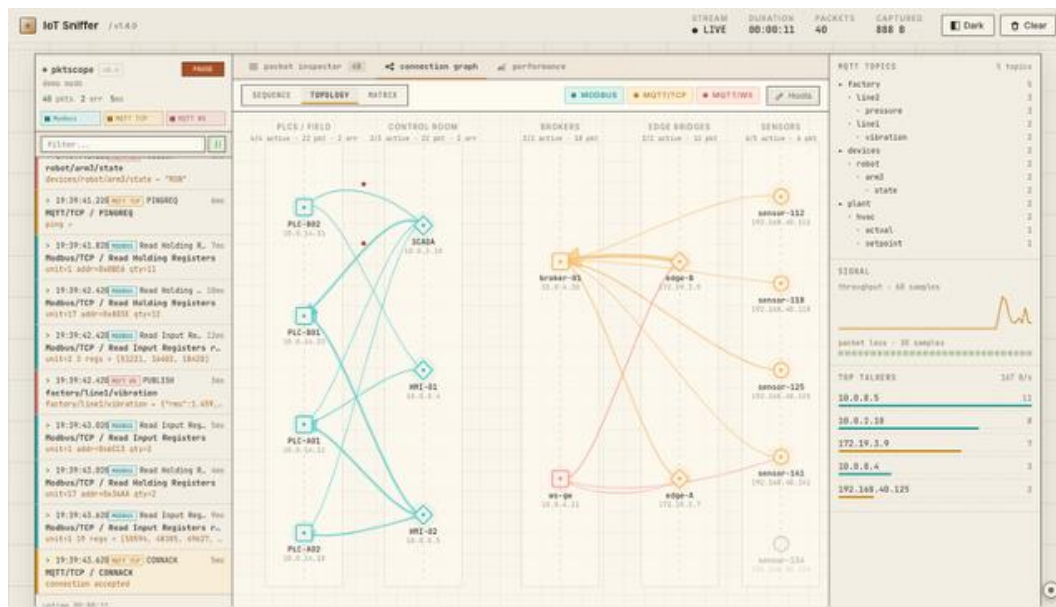
ง.3 มุมมองแคชบอร์ด

- ตัวตรวจสอบแพ็กเก็ต - แดชบอร์ดที่ระบายสีตามโปรโตคอล การคลิกที่เฟรมจะขยายแผนผังฟิลด์ที่แสดงช่วงไบนารีของ MBAP และ PDU แต่ละส่วนพร้อมค่าที่ถอดรหัสแล้ว (ตัวอย่างการถอดรหัสแสดงแล้วในภาพที่ 3.11)
- มุมมองลำดับ - แผนภาพลำดับข้อความสเตล UML พร้อมเส้นชีวิตของอุปกรณ์และเวลาที่ไหลลงด้านล่าง ใช้ยืนยันการจับคู่คำขอ/การตอบสนอง ดังภาพที่ ง.1

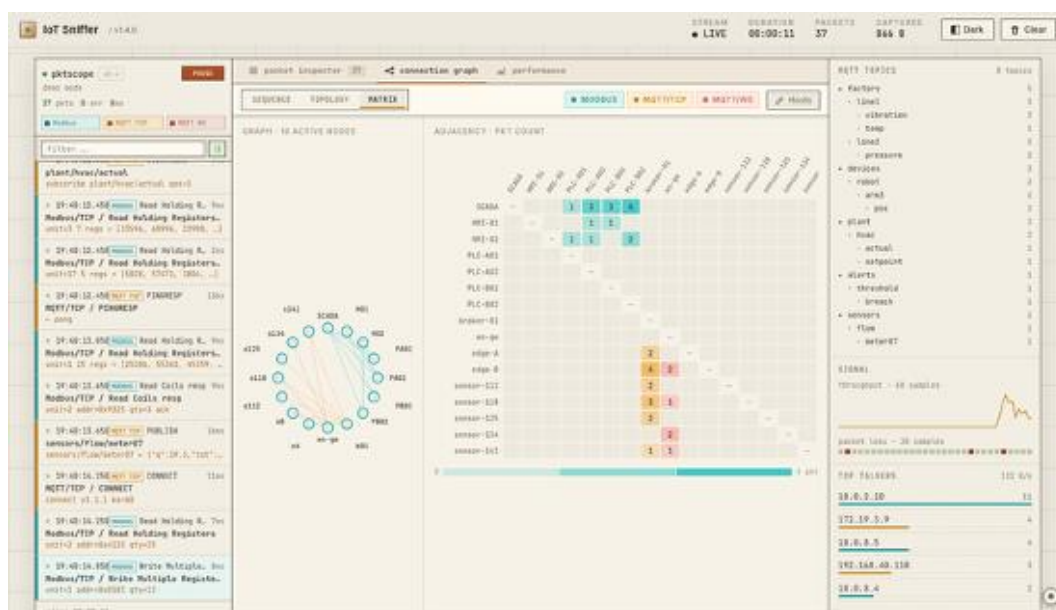


ภาพที่ ง.1 มุมมองลำดับ (sequence view) แสดงการจับคู่คำขอและการตอบกลับ Modbus TCP ระหว่างอุปกรณ์บนเส้นเวลา

- มุมมองโทโพโลยีและเมตริกซ์ - กราฟระหว่างอุปกรณ์กับอุปกรณ์ โดยน้ำหนักเส้นเชื่อมปรับตามจำนวนแพ็กเก็ต ใช้ดูว่าโหนดใดมีปริมาณข้อมูลมากที่สุดในการดักจับ ดังภาพที่ ง.2 และภาพที่ ง.3

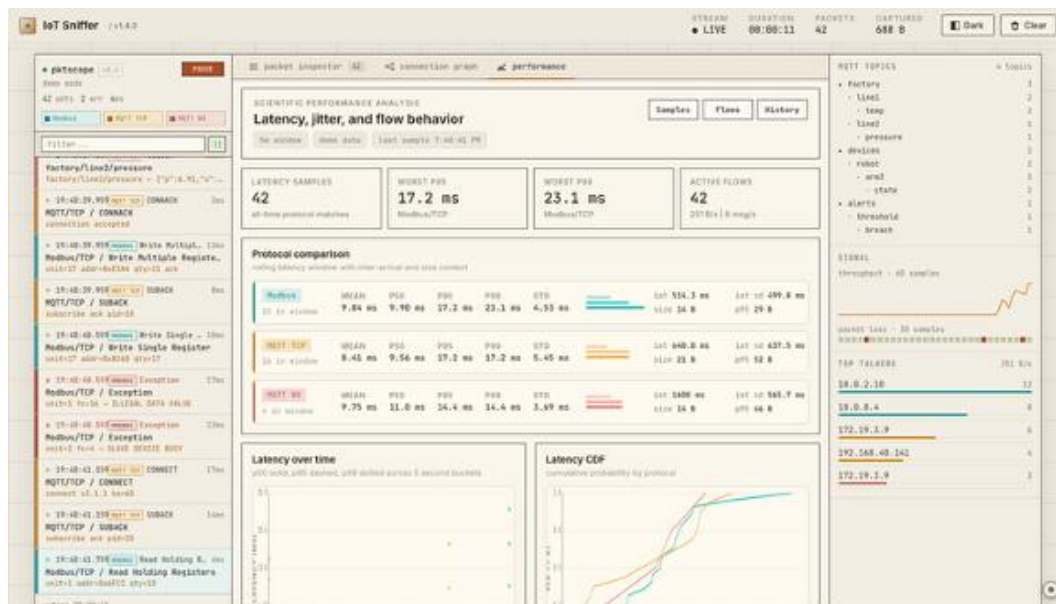


ภาพที่ ง.2 มุมมองโทโปโลยี (topology view) แสดงการเชื่อมต่อระหว่างอุปกรณ์ในเครือข่าย
แยกตามกลุ่มอุปกรณ์ทำงาน ห้องควบคุม โบรกเกอร์ เอจจ์ และเซนเซอร์



ภาพที่ ง.3 มุมมองเมทริกซ์ (matrix view) แสดงจำนวนแพ็กเก็ตที่แลกเปลี่ยนกันระหว่างอุปกรณ์
แต่ละคู่

- แท็บสมรรถนะ - เวลาแฝงที่ p50/p95/p99 จิตเตอร์ของช่วงเวลามาถึง (inter-arrival jitter) และการกระจายขนาดเฟรมต่อโปรโตคอล พร้อมส่งออก CSV ด้วยคลิกเดียวสำหรับการวิเคราะห์แบบออฟไลน์ ดังภาพที่ ง.4



ภาพที่ ง.4 แท็บสมรรถนะแสดงเวลาแฝงที่ p50/p95/p99 จิตเตอร์ และพฤติกรรมของไฟล์แยกตามโปรโตคอล

ภาคผนวก จ

ข้อมูลอ้างอิงฮาร์ดแวร์

ภาคผนวกนี้รวบรวมข้อกำหนดสำคัญทางไฟฟ้า เชนก และการสื่อสารของฮาร์ดแวร์ที่ใช้ในแพลตฟอร์ม ส่วนภาพประกอบที่อ้างถึงในภาคผนวกนี้ปรากฏอยู่ในสารบัญภาพ

จ.1 **Waveshare ESP32-S3-Relay-6CH (MCU)**

บอร์ดรีเลย์สำหรับงานอุตสาหกรรมขนาดกะทัดรัดที่มีโมดูล Espressif ESP32-S3-WROOM-1 เป็นแกนหลัก ถูกเลือกใช้เป็นเอดจ์เกตเวย์ เนื่องจากมีตัวรับส่งสัญญาณ RS-485 มาในตัว มีรีเลย์หกช่อง มี PSRAM และรองรับ PlatformIO

- SoC: ESP32-S3 (Xtensa LX7 สองแกนประมวลผล สูงสุด 240 MHz), Wi-Fi 802.11 b/g/n + BLE 5
- หน่วยความจำ: แฟลช Quad-SPI 16 MB + PSRAM 8 MB, SRAM ในชิป 512 KB
- รีเลย์: 6 x SPDT, 250 VAC/10 A หรือ 30 VDC/10 A ขับเคลื่อนผ่าน GPIO ด้วยการแยกทางไฟฟ้าแบบออปโตคัปเปิลอร์
- บัสหน้างาน: ตัวรับส่งสัญญาณ RS-485 (half-duplex) ในตัวบน UART ฮาร์ดแวร์
- แหล่งจ่ายไฟ: 7-36 V DC ผ่านขั้วต่อสกรู หรือ 5 V ผ่าน USB-C (ใช้สำหรับแฟลชโปรแกรม/ดีบั๊กด้วย) ลอจิก 3.3 V
- การใช้พลังงาน: ประมาณ 0.6-1.7 W ขึ้นกับการะของรีเลย์
- ขนาด: 65 x 56 มม.; ชุดเครื่องมือ: Arduino core / ESP-IDF ผ่าน PlatformIO



ภาพที่ จ.1 บอร์ด Waveshare ESP32-S3-Relay-6CH (ที่มา: waveshare.com)

จ.2 Siemens LOGO! 24CE - เฟิร์มแวร์ V8.4 (PLC)

ไมโครพียูแอลซีขนาดเล็กที่ติดตั้งที่ใช้ไฟ 24 V DC พร้อมเซิร์ฟเวอร์ Modbus TCP ในตัว เขียนโปรแกรมได้เฉพาะผ่านซอฟต์แวร์ LOGO! Soft Comfort ของผู้ผลิต ด้วยแผนภาพแลดเดอร์หรือแผนภาพบล็อกฟังก์ชันตามมาตรฐาน IEC 61131-3

- รหัสสินค้า: 6ED1052-1CC08-0BA2; แหล่งจ่ายไฟ 24 V DC
- I/O: อินพุตดิจิทัล 24 V DC จำนวน 8 ช่อง (I1-I4 ใช้เป็นแอนะล็อก 0-10 V ได้), เอาต์พุตทรานซิสเตอร์ 24 V DC / 0.3 A จำนวน 4 ช่อง
- เครือข่าย: อีเทอร์เน็ต 10/100 Mbit/s จำนวน 1 พอร์ต; เซิร์ฟเวอร์และไคลเอนต์ Modbus TCP บนพอร์ต 502 รองรับสูงสุด 8 การเชื่อมต่อ
- ฟังก์ชัน Modbus: FC01/02/03/05/06/15/16; word ใน V-memory ถูกแมปเป็น holding registers, coils ถูกแมปเป็น Q1-Q4
- หน่วยความจำ: ฟังก์ชันบล็อก 400 บล็อก, V-memory แบบคงค่า 4,000 ไบต์; นาฬิกาเรียลไทม์สำรองด้วยแบตเตอรี่
- การรับรองมาตรฐาน: CE, UL, cUL, FM, ATEX Zone 2; ติดตั้งบนราง DIN; ขนาด 72 x 90 x 55 มม.

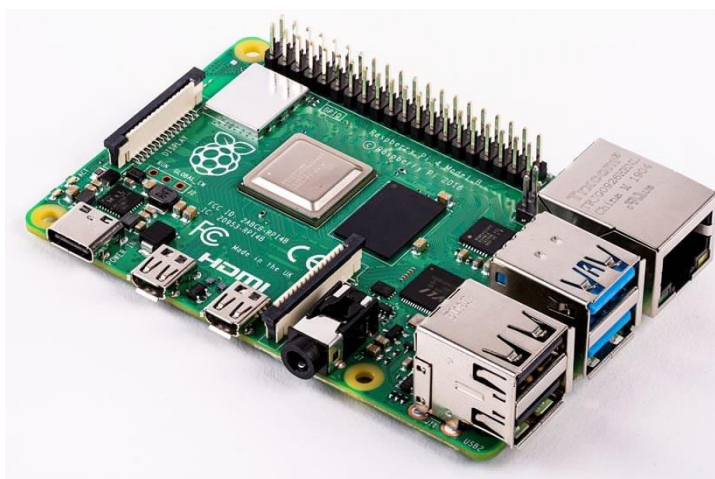


ภาพที่ จ.2 พีแอลซี Siemens LOGO! 24CE (ที่มา: siemens.com)

จ.3 Raspberry Pi 4 Model B, 2 GB (SBC)

เครื่องโฮสต์ของชั้นไอทีที่รันชุด TIG และ Mosquitto อยู่ภายใต้ Docker Compose ทำหน้าที่ขับเคลื่อนจอแสดงผลแบบคีย์บอร์ดของ Grafana

- SoC: Broadcom BCM2711 (สี่แกนประมวลผล Cortex-A72, 1.5 GHz); แรม: 2 GB LPDDR4
- เครือข่าย: กิกะบิตอีเทอร์เน็ต; ระบบปฏิบัติการ: Raspberry Pi OS Lite 64 บิต (Debian Bookworm)
- แหล่งจ่ายไฟ: 5.1 V / 3 A ผ่าน USB-C; ปกติ ~3.4 W, สูงสุด ~7.6 W



ภาพที่ จ.3 Raspberry Pi 4 Model B (2 GB) (ที่มา: raspberrypi.com)

จ.4 โมดูลเซนเซอร์วัดอุณหภูมิและความชื้นสัมพัทธ์ XY-MD02 (เครื่องมือวัดหน้างาน)

โมดูลเซนเซอร์วัดอุณหภูมิและความชื้นสัมพัทธ์แบบติดตั้งบนรางปีกนก (DIN Rail) ที่ใช้เซนเซอร์ Sensirion SHT20 เป็นอุปกรณ์หน้างานที่สื่อสารด้วย Modbus RTU เพียงอย่างเดียว ไม่มีชุดโปรโตคอลไอพีในตัว

- การตรวจวัด: SHT20; อุณหภูมิ ± 0.3 องศาเซลเซียส, ความชื้น $\pm 3\%$ RH
- โปรโตคอล: Modbus RTU ผ่านบัส RS-485, 9,600 baud 8-N-1, หมายเลขอุปกรณ์สเลฟ (slave address) 0x01
- รีจิสเตอร์ (FC=0x04): อุณหภูมิ 10 และความชื้น 10 เป็นจำนวนเต็ม 16 บิต
- แหล่งจ่ายไฟ: 10-30 V DC, < 30 mA



ภาพที่ จ.4 โมดูลเซนเซอร์วัดอุณหภูมิและความชื้น XY-MD02 (ที่มา: miniature-solution.com)

จ.5 เพาเวอร์มิเตอร์ไฟฟ้าเฟสเดียว Eastron SDM230-Modbus (เครื่องมือวัดหน้างาน)

เพาเวอร์มิเตอร์ไฟฟ้าดิจิทัลแบบเฟสเดียว (Single-Phase Digital Power Meter) รุ่น Eastron SDM230-Modbus แบบต่อตรง (direct connection) รองรับกระแสสูงสุด 100 A ติดตั้งบนรางปีกนก (DIN Rail) ขนาดสองโมดูล พร้อมจอแสดงผล LCD แบบมีไฟส่องหลัง ให้ค่าการวัดทางไฟฟ้าจริงในระดับอุตสาหกรรมและส่งออกเป็นรีจิสเตอร์ทศนิยม IEEE-754

- การวัด (15 พารามิเตอร์): แรงดันไฟฟ้า กระแสไฟฟ้า ความถี่ ค่าตัวประกอบกำลังไฟฟ้า (Power Factor) กำลังไฟฟ้าที่ใช้งานจริง (Real Power) กำลังไฟฟ้าปรากฏ (Apparent

Power) กำลังไฟฟ้รีแอกทีฟ (Reactive Power) และพลังงานไฟฟ้าที่นำเข้า/ส่งออก/รวม (kWh, kVArh)

- การเข้ารหัส: ค่าทศนิยม IEEE-754 ขนาด 32 บิต ในคู่ input register (ลำดับ word แบบ big-endian)
- โพรโทคอล: Modbus RTU ผ่านบัส RS-485 (baud 2,400/4,800/9,600); ESP32-S3 อ่านค่าด้วย FC=0x03/0x04 พร้อมเอาต์พุตพัลส์ (pulsed output) สำหรับพลังงาน
- พิกัด: เฟสเดียว 230 V ต่อตรงสูงสุด 100 A; ผ่านการรับรอง MID Class B (EN50470-3); ติดตั้งบนรางปีกนขนาดสองโมดูล



ภาพที่ จ.5 เพาเวอร์มิเตอร์ไฟฟ้าเฟสเดียว Eastron SDM230-Modbus (ที่มา: eastroneurope.com)

ภาคผนวก จ

ตารางเปรียบเทียบแพลตฟอร์มไอโอโอที

ภาคผนวกนี้นำตัวควบคุมทั้งสามประเภทของแพลตฟอร์มมาเปรียบเทียบกันในหกมิติเชิงวิศวกรรม โดยขยายความจากตารางสรุปในตารางที่ 5.1 ทั้งนี้ไมโครคอนโทรลเลอร์ทางเลือกอื่นที่ได้สำรวจแต่ไม่ได้นำมาใช้ (เช่น STM32 ตระกูล F7 และ Arduino Mega) อยู่นอกขอบเขตของการเปรียบเทียบนี้ ตารางด้านล่างนี้จำกัดเฉพาะอุปกรณ์ที่ใช้งานจริง เพื่อให้ทุกคะแนนอ้างอิงจากระบบที่สร้างขึ้นจริงในงานนี้ เกณฑ์การให้คะแนน: Full = รองรับได้เต็มรูปแบบในตัว, Part = รองรับบางส่วนหรือต้องใช้อุปกรณ์เสริม, None = ไม่รองรับ

จ.1 ตารางเปรียบเทียบ (MCU เทียบกับ SBC เทียบกับ PLC)

ตารางที่ จ.1 การเปรียบเทียบตัวควบคุม MCU, SBC และ PLC ในหกมิติเชิงวิศวกรรม

มิติ	ESP32-S3 (MCU)	RPi 4 (SBC)	LOGO! 24CE (PLC)
Modbus RTU (RS-485)	เต็มรูปแบบ (ในตัว)	บางส่วน (ผ่านอะแดปเตอร์ USB)	เต็มรูปแบบ (ในตัว)
Modbus TCP	เต็มรูปแบบ (Wi-Fi/IP)	เต็มรูปแบบ (สแตก Linux)	เต็มรูปแบบ (เซิร์ฟเวอร์ในตัว)
MQTT	เต็มรูปแบบ (ไคลเอนต์ในตัว)	เต็มรูปแบบ (โฮสต์โบรกเกอร์)	ไม่มี
ความแน่นอนเชิงเวลา	เรียลไทม์แบบไม่เข้มงวด	ไม่มีความแน่นอนเชิงเวลา	เรียลไทม์แบบเข้มงวด
I/O หน่วยงาน	GPIO/ADC/UART/RS-485	GPIO; ไม่มีแอนะล็อก	DI 24 V, AI 0-10 V, รีเลย์
ชุดเครื่องมือ	เปิด (Arduino/PIO)	เปิด (Linux/Docker)	เฉพาะผู้ผลิต (Soft Comfort)
การรับรองมาตรฐาน	ไม่มี	ไม่มี	CE/UL/FM/ATEX
ต้นทุนโดยประมาณ	~1,895 บาท	~1,900-3,200 บาท	~5,000-6,500 บาท

จ.2 การอ่านตาราง

ไม่มีอุปกรณ์ประเภทใดที่เหนือกว่าในทุกด้าน MCU โค้ดเด่นเรื่องต้นทุนต่อช่องสัญญาณ ส่วนต่อประสานหน้างานที่มีมาให้ในตัว และชุดเครื่องมือแบบเปิด แต่ขาดระบบปฏิบัติการและการรับรองมาตรฐาน ส่วน SBC โค้ดเด่นเรื่องการประมวลผลที่ขอบเครือข่ายและความหลากหลายของซอฟต์แวร์ แต่ไม่สามารถรับประกันการทำงานแบบเรียลไทม์อย่างเข้มงวดได้ ขณะที่ PLC เป็นเพียงอุปกรณ์เดียวที่ให้การควบคุมที่ผ่านการรับรองด้วยรอบสแกน ที่มีขอบเขตแน่นอนสำหรับ แอ็กชูเอเตอร์ระดับแรงดันไฟหลัก แต่ก็มีความสูงที่สุดและปิดที่สุด ตารางนี้จึงเป็นหลักฐานสนับสนุนข้อเสนอของปริญญานิพนธ์ที่ว่าตัวควบคุมแต่ละประเภทต่างมีจุดแข็งของตนเอง และระบบจะสมบูรณ์ยิ่งขึ้นเมื่อนำทั้งสามประเภทมาทำงานร่วมกัน